



**COMSATS University Islamabad,
Sahiwal Campus**

**“Khao Piyoo”
Universal Food App**

**A Project Presented To
COMSATS University Islamabad, Sahiwal Campus**

**In partial fulfillment
of the requirement for the degree of**

Bachelor of Science in Computer Science (2022-2026)

By

**Abdullah Rafiq
Malik Ali Raza**

**CIIT/FA22-BCS-006/SWL
CIIT/FA22-BCS-045/SWL**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Abdullah Rafiq

Malik Ali Raza

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) “**Khao Piyoo**” was developed by **Abdullah Rafiq (CIIT/FA22-BCS-006/SWL)** and **Malik Ali Raza (CIIT/FA22-BCS-045/SWL)** under the supervision of “**Dr. Zafar Iqbal Roy**” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Executive Summary

Khaoo Piyoo is a food discovery and ordering platform designed to make restaurant selection easier, more transparent, and more convenient for users. The system provides a single digital platform where users can explore restaurants, view menus, compare prices, check locations, and place orders without depending on multiple applications or scattered sources of information.

The project addresses the common problem faced by users who want to know restaurant menus, prices, ratings, and nearby availability before making a decision. It is especially useful for budget-conscious users and people visiting unfamiliar areas. The system also supports restaurants by giving them a platform to register their business, manage menus, upload offers, and improve their digital visibility.

The application includes important modules such as user authentication, restaurant discovery, menu management, cart and checkout, AI-based assistance, reels engagement, loyalty rewards, and admin management. The AI assistant helps users receive food or restaurant suggestions based on their cravings, budget, and preferences, while the reels section increases user engagement through short food-related videos.

The system is developed using modern mobile application technologies with cloud-based backend services for data storage, authentication, media handling, and real-time updates. Testing results show that the major modules of the system work according to the defined requirements, including login, restaurant search, cart update, checkout, AI assistant, reels, and profile management.

Overall, **Khaoo Piyoo** provides a practical and user-friendly solution for improving food discovery and restaurant visibility. The system can be further enhanced in the future with online payments, real-time delivery tracking, advanced recommendations, restaurant analytics, and multilingual support.

Acknowledgement

All praise is to Almighty **Allah** who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task. We are greatly indebted to our project supervisor “**Dr. Zafar Iqbal Roy**”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work and we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Abdullah Rafiq

Malik Ali Raza

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
DB	Database
FYP	Final Year Project
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
NLP	Natural Language Processing
OOP	Object-Oriented Programming
PC	Personal Computer
REST	Representational State Transfer
SDK	Software Development Kit
SDD	Software Design Description
SDLC	Software Development Life Cycle
SRS	Software Requirements Specification
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
OTP	One-Time Password
NoSQL	Not Only SQL
SSL	Secure Sockets Layer
TLS	Transport Layer Security

Table of Contents

1	Introduction.....	2
1.1	Brief	2
1.2	Relevance to Course Modules	3
1.2.1	Software Engineering Concepts (CSC291)	3
1.2.2	Database Systems (CSC270)	3
1.2.3	Mobile Application Development (CSC303)	4
1.2.4	Artificial Intelligence (CSC462).....	4
1.3	Project Background.....	4
1.4	Literature Review.....	5
1.5	Analysis from Literature Review	6
1.6	Methodology and Software Lifecycle.....	7
1.6.1	Key phases	7
1.6.2	Rationale behind Selected Methodology	8
2	Problem Definition.....	10
2.1	Problem Statement	10
2.2	Deliverables and Development Requirements	11
2.2.1	Software Deliverables	12
2.2.2	Documentation Deliverables.....	12
2.2.3	Development Requirements.....	12
2.3	Current System.....	13
3	Requirement Analysis	16
3.1	Use Case Diagram.....	16
3.2	Detailed Use Case	23
3.2.1	Use Case Description	23
3.3	Functional Requirements	27
3.3.1	User Registration and Authentication	27
3.3.2	Restaurant Registration and Profile Management	28
3.3.3	Menu Management	28
3.3.4	Restaurant Discovery and Search	28
3.3.5	Location-Based Restaurant Suggestions.....	28
3.3.6	Online Ordering and Cart Management.....	29
3.3.7	AI Chatbot and Recommendation System.....	29

3.3.8	Food Reels and Content Engagement	29
3.3.9	Offers, Promotions, and Loyalty Rewards.....	29
3.3.10	Admin Management.....	29
3.4	Non-Functional Requirements	30
3.4.1	Performance	30
3.4.2	Usability.....	30
3.4.3	Security	30
3.4.4	Reliability.....	30
3.4.5	Scalability	31
3.4.6	Maintainability.....	31
3.4.7	Compatibility	31
3.4.8	Availability	31
4	Design and Architecture	33
4.1	System Architecture.....	33
4.2	Data Representation [Diagram + Description]	34
4.3	Process Flow/Representation	35
4.4	Design Models	37
4.4.1	Class Diagram.....	38
4.4.2	Sequence Diagram	39
4.4.3	State Transition Diagram	40
4.4.4	Data Flow Diagram.....	41
5	Implementation	44
5.1	Algorithm.....	45
5.1.1	User Registration and Login Algorithm	45
5.1.2	Restaurant Discovery Algorithm	46
5.1.3	Menu and Cart Management Algorithm.....	46
5.1.4	Order Placement Algorithm.....	47
5.1.5	AI Recommendation Algorithm	48
5.1.6	Reels Display Algorithm.....	49
5.1.7	Loyalty Token Algorithm	49
5.1.8	Admin Management Algorithm.....	50
5.2	External APIs.....	51
5.3	User Interface.....	51
5.3.1	Landing Page / Website Interface	52
5.3.2	Role Selection Screen	53

5.3.3	User Profile Screen	54
5.3.4	Reels / Content Screen	55
5.3.5	AI Assistant Interface	56
5.3.6	Cart Screen.....	57
6	Testing and Evaluation	60
6.1	Manual Testing	61
6.1.1	System testing	61
6.1.2	Unit Testing	62
6.1.3	Functional Testing	63
6.1.4	Automated Testing.....	65
7	Conclusion and Future Work	68
7.1	Conclusion	69
7.2	Future Work.....	69
	References.....	71

List of Figures

Figure 3.1 Use case diagram Restaurant owner	18
Figure 3.2 Use case diagram Customer Side	19
Figure 3.3 Use Case diagram Content creator	20
Figure 3.4 Use Case Diagram Ai Chatbot	21
Figure 3.5 Use Case Diagram Admin Side	22
Figure 4.1 Data Representation Diagram.....	35
Figure 4.2 Process Flow Diagram.....	37
Figure 4.3 Class Diagram	39
Figure 4.4 Sequence Diagram.....	40
Figure 4.5 State Transition Diagram.....	41
Figure 4.6 Data Flow Diagram	42
Figure 5.1 Landing Page / Website Interface.....	52
Figure 5.2 Landing Page / Website Interface.....	53
Figure 5.3 Role Selection.....	53
Figure 5.4 User Profile Screen.....	54
Figure 5.5 Reels / Content Screen.....	55
Figure 5.6 AI Assistant Interface	56
Figure 5.7 Cart Screen	57

List of Tables

Table 2.1 Existing System Limitations.....	14
Table 3.1 Browse Restaurants & Menus (Customer)	23
Table 3.2 Order Placing	24
Table 3.3 Manage Menu & Prices (Restaurant).....	25
Table 3.4 Manage Menu & Prices (Restaurant).....	25
Table 3.5 Upload Food Reels (Content Creator)	25
Table 3.6 Upload Food Reels (Content Creator)	26
Table 3.7 Recommend Meals & Restaurants (AI Bot).....	26
Table 5.1 Details of APIs Used in the Project	51
Table 6.1 Manual Testing Results	61
Table 6.2 Login Testing.....	63
Table 6.3 Edit Profile Testing.....	63
Table 6.4 Login with Different Roles	64
Table 6.5 Restaurant Search and Menu Viewing.....	64
Table 6.6 Cart and Checkout Testing	65
Table 6.7 AI Assistant and Reels Testing.....	65
Table 6.8 Automated Testing Tools and Results	66

Chapter 1

Introduction

1 Introduction

The food industry has undergone significant transformation with the advancement of digital technologies and the widespread use of smartphones. People increasingly rely on mobile applications to explore dining options, check restaurant details, and make food-related decisions. However, despite the availability of multiple platforms, users still face difficulties in accessing complete and reliable information about restaurants, menus, pricing, and nearby options in one place.

The proposed system “**Khaoo Piyoo**” is designed to provide a unified platform that simplifies food discovery and ordering. It allows users to explore restaurants, view detailed menus, compare prices, and make decisions based on their preferences and budget. By integrating location-based services, the system helps users identify nearby restaurants and access navigation support, making it especially useful when traveling or visiting unfamiliar areas.

In addition to basic functionality, the system introduces intelligent features to enhance user experience. An AI-based assistant is incorporated to help users select meals and restaurants according to their cravings, previous choices, and financial constraints. This reduces confusion and improves decision-making by providing personalized suggestions. Furthermore, the platform includes a content-driven section where food-related videos and promotional material can be shared, increasing user engagement and interaction.

The system also benefits restaurants by providing them with a digital platform to showcase their offerings. Restaurants can register, upload menus, promote deals, and reach a wider audience without relying solely on traditional marketing methods. This creates a balanced environment where both users and businesses can benefit from the platform.

Overall, the project focuses on improving accessibility, transparency, and convenience in the food industry by combining restaurant discovery, intelligent recommendations, and user engagement features into a single application.

1.1 Brief

The project presents the development of a comprehensive food discovery and ordering platform that aims to simplify the process of selecting restaurants and meals. The system provides users with access to detailed restaurant information, including menus, pricing, location, ratings, and available offers, enabling them to make informed decisions based on their preferences and budget. In addition to basic browsing and ordering features, the platform integrates intelligent

assistance through an AI-based chatbot that helps users choose suitable options according to their needs and cravings.

The outcome of this project is a fully functional mobile application that supports restaurant registration, menu management, location-based search, order placement, and user engagement through promotional content such as short food videos. The system is designed to serve both customers and restaurants by creating a centralized digital environment where users can explore options easily, while businesses can improve their visibility and attract more customers.

The development of the system utilizes modern technologies, including cross-platform mobile development frameworks for building the user interface and cloud-based services for data storage and real-time synchronization. The project follows an object-oriented design approach, ensuring modularity and scalability, while the overall development process is based on an incremental model, allowing the system to be built and improved in phases.

This report is structured to provide a complete understanding of the project. It begins with an introduction and background, followed by problem definition and requirement analysis. The design and architecture of the system are then explained in detail, along with implementation techniques and tools used. Finally, the report includes testing, evaluation, and conclusions, highlighting the effectiveness of the system and potential areas for future improvement.

1.2 Relevance to Course Modules

The development of this project is closely related to various courses studied during the Bachelor of Science in Computer Science program, as it applies both theoretical knowledge and practical skills in a real-world system.

1.2.1 Software Engineering Concepts (CSC291)

The concepts of Software Engineering are applied in requirement analysis, system design, and the use of a structured development process. The preparation of system documents and the use of a development model reflect the principles of planning, designing, and managing a software project effectively.

1.2.2 Database Systems (CSC270)

Knowledge from Database Systems is used in designing and managing data related to users, restaurants, menus, and orders. Efficient data storage and retrieval are important for ensuring system performance and reliability.

1.2.3 Mobile Application Development (CSC303)

The project also relates to Mobile Application Development, where user interfaces are designed to be simple, responsive, and easy to use. The application is developed to work smoothly on mobile devices, ensuring a good user experience.

1.2.4 Artificial Intelligence (CSC462)

Concepts from Artificial Intelligence are applied using an AI-based assistant that helps users make decisions by providing recommendations based on their preferences and inputs.

- Additionally, Computer Networks and Web Technologies are involved in connecting the application with backend services, enabling data communication, real-time updates, and online functionalities.
- Overall, the project integrates multiple course concepts to develop a complete and practical system, demonstrating how academic knowledge can be applied to solve real-world problems.

1.3 Project Background

The idea behind this project is to develop a unified platform that simplifies the process of discovering restaurants and selecting food. In the current environment, users often rely on multiple sources to gather information, such as searching for nearby restaurants, checking reviews, and exploring menus on different platforms. This scattered approach makes decision-making time-consuming and sometimes confusing, especially when complete information is not available in one place.

One of the main problems faced by users is the lack of transparency regarding menu details and pricing. Many people prefer to know what a restaurant offers before visiting, particularly when they are concerned about budget or trying a new place. This issue becomes more significant when users are in unfamiliar locations, where they have limited knowledge about available restaurants and their offerings. From the restaurant's perspective, especially for small or newly established businesses, gaining visibility is a major challenge. Many restaurants do not have access to effective digital platforms to promote their menus and offers, which limits their ability to attract customers. Existing applications often focus only on selected or partnered restaurants, leaving out many potential options.

To address these challenges, the project is based on the concept of creating a centralized system where all restaurants can register and display their menus, prices, and location details. Users can explore and compare different options within a single platform, making the process easier and more efficient. The system also integrates location-based services to help users find nearby restaurants quickly and accurately. In addition, an AI-based assistant is included to guide users in selecting meals and restaurants according to their preferences, budget, and needs. This makes the platform more interactive and personalized.

Overall, the project aims to provide a complete solution that improves accessibility, transparency, and user experience by combining restaurant discovery, intelligent recommendations, and user engagement features into one system.

1.4 Literature Review

The food industry has rapidly adopted digital technologies, leading to the development of various applications that assist users in exploring restaurants and ordering food. Many existing platforms focus on online food delivery services, allowing users to browse restaurants, place orders, and track deliveries. These systems have improved convenience but are often limited to partnered restaurants, which restricts the range of available options and reduces transparency for users.

Another important trend is the use of location-based services, where applications provide nearby restaurant suggestions based on the user's current location. These systems help users quickly find available options, especially in unfamiliar areas. However, most of these platforms provide only basic information such as ratings and directions, without offering complete menu details or comparison features, which are important for making informed decisions.

In recent years, artificial intelligence has been increasingly used in applications to enhance user experience. Recommendation systems analyze user preferences and behavior to suggest relevant options, reducing the effort required to search for suitable choices. Similarly, AI-based chatbots are used to assist users by answering queries and providing suggestions in a conversational manner. Despite their advantages, many food-related applications have not fully integrated these intelligent features into their systems. Another emerging trend is the use of content-based engagement, particularly through short videos and visual media. Social media platforms have shown that users are highly influenced by visual content when making decisions related to food. However, most food applications do not include such features, limiting user interaction and engagement.

Overall, the analysis of existing systems shows that while current applications provide useful functionalities, they lack integration of key features such as complete restaurant discovery, menu comparison, intelligent recommendations, and content engagement. This project aims to address these limitations by combining all these aspects into a single, unified platform.

1.5 Analysis from Literature Review

The literature review shows that existing food-related platforms provide useful services, but most of them focus on limited areas of the food experience. Food delivery applications mainly support ordering and delivery, while location-based applications mostly help users find nearby restaurants and directions. These systems are helpful, but they do not fully solve the problem of restaurant discovery, price comparison, personalized suggestions, and user engagement in one place.

In comparison, the proposed system combines multiple features into a single platform. Instead of focusing only on delivery, it supports users in exploring restaurants, checking menus, comparing prices, viewing location details, and making better decisions before ordering or visiting. This makes the system complete and more useful for users who want transparency and convenience.

The analysis of existing systems highlights the following gaps:

- Most platforms only show partnered or selected restaurants.
- Complete menu and price comparison is often missing.
- AI-based personal recommendations are limited or unavailable.
- Visual engagement through food reels is not commonly integrated.
- Small and local restaurants receive limited digital exposure.
- Users often need multiple apps to gather complete information.

The proposed system addresses these gaps by providing restaurant discovery, AI-based assistance, location-based search, menu visibility, promotional content, and ordering features within one application. The AI assistant improves personalization by helping users choose restaurants or meals according to their cravings, budget, and preferences. The reels section adds a social and visual element, making the platform more engaging than traditional food applications.

Overall, the comparison shows that the proposed work is not only an ordering system but a broader food discovery and engagement platform. It aims to improve user confidence, support restaurants, and provide a more organized digital solution for the food industry.

1.6 Methodology and Software Lifecycle

The project follows an Object-Oriented Software Development Methodology because the system is based on different interacting modules such as users, restaurants, menus, orders, AI assistant, reels, loyalty rewards, and admin management. Each module can be represented as an object with its own attributes, functions, and responsibilities. This approach makes the system easier to design, implement, test, and maintain. Object-oriented methodology is suitable for this project because it supports modular development. For example, the customer module handles restaurant browsing and ordering, the restaurant module manages menus and promotions, while the AI module provides recommendations. These modules are connected with each other but can still be developed and improved separately. This helps reduce complexity and makes future updates easier.

The Incremental Model is suitable because the project contains several major features that require separate testing and gradual integration. After completing each increment, the module can be tested, reviewed, and improved before moving to the next part. This reduces the chances of major errors and allows the development team to manage the project more effectively.

Overall, the combination of Object-Oriented Methodology and Incremental Software Lifecycle provides a practical and structured approach for this project. It supports flexibility, reusability, maintainability, and scalability, which are important for developing a food discovery and ordering platform with AI-based features

1.6.1 Key phases

1. Requirement Analysis
2. UI/UX Prototyping
3. ML Model Training
4. API Development
5. Mobile Application Development
6. Integration Testing
7. Deployment
8. User Feedback Collection.

1.6.2 Rationale behind Selected Methodology

The Object-Oriented Methodology is selected because the system consists of multiple real-world entities that interact with each other, such as customer, restaurant, menu, order, AI assistant, reels, loyalty tokens, and admin. These entities can be clearly represented as objects with specific attributes and functions. This makes the system design more organized and easier to understand.

This methodology is also suitable because the project is modular in nature. Each major feature can be developed as a separate module, while still remaining connected with the overall system. For example, restaurant management, menu management, order placement, AI recommendations, and admin control can be designed independently and then integrated together. This improves code reusability, reduces complexity, and makes future maintenance easier.

The Incremental Model is selected as the software lifecycle because the complete system cannot be developed efficiently in one step. The project includes several important features, so it is better to develop them in smaller working parts. Basic features such as authentication and restaurant listing can be completed first, followed by menu management, ordering, AI chatbot, reels, and admin dashboard.

This model also allows testing after each increment, which helps identify errors early and improves system quality. It provides flexibility to add improvements during development, especially in features like AI recommendations and user engagement, where feedback may be needed. Therefore, the selected methodology and lifecycle model are appropriate for building a scalable, maintainable, and user-friendly application. The Incremental Model also provides flexibility during development. If any improvement is required in features such as AI recommendations, user engagement, or restaurant management, it can be adjusted in the relevant increment without affecting the whole project. Therefore, the selected methodology and lifecycle model are appropriate for building a scalable, maintainable, and user-friendly application.

Chapter 2

Problem Definition

2 Problem Definition

The food industry is large and continuously growing, but restaurant information is still scattered across different platforms. Users often struggle to find complete details about nearby restaurants, including menus, prices, ratings, location, and ordering options. Because of this, selecting a suitable restaurant becomes difficult, especially when users want to compare options before visiting or placing an order.

Many existing platforms focus mainly on food delivery or basic restaurant search. They do not provide a complete solution where users can view all available restaurants, compare meals, check price ranges, and make decisions according to their budget and preferences. This creates uncertainty for customers, particularly for middle-class users who prefer to know menu prices beforehand. The problem is also faced by travelers or users visiting unfamiliar areas. They may not know which restaurants are available nearby or which one is suitable according to their taste and budget. Without proper guidance, users may depend on random searches, incomplete reviews, or personal guesses.

Restaurants also face difficulty in reaching customers digitally. Small and newly opened restaurants often lack visibility because they are not properly represented on major platforms. As a result, they miss opportunities to attract customers, promote deals, and grow their business. The outcome of the proposed system is to provide a centralized platform where users can discover restaurants, view menus, compare prices, access location-based options, and receive AI-assisted recommendations. At the same time, restaurants can register, manage their information, promote offers, and improve their digital presence.

2.1 Problem Statement

The food industry is widely growing, but there is still no proper centralized system that provides complete and reliable information about restaurants, menus, pricing, and location-based availability in one place. Users often face difficulty when they want to choose a restaurant because they do not have access to all necessary details before visiting or ordering. This creates confusion and hesitation, especially for users who want to make decisions based on their budget and preferences.

Most existing applications focus only on specific features such as food delivery or basic restaurant searching. These systems do not allow proper comparison between restaurants, nor do they provide complete menu visibility and price transparency. As a result, users are forced

to rely on multiple platforms, social media, or personal experience to gather information, which makes the decision-making process time-consuming and inefficient.

This problem becomes more significant in real-life situations such as:

- When a user wants to try a new restaurant but is unsure about menu prices and food quality
- When a person is in a new city and does not know which restaurants are available nearby
- When users want to compare multiple restaurants before making a final decision
- When users depend on incomplete or unreliable online information

In addition to user-related issues, restaurants also face challenges in reaching customers. Small and newly established restaurants often do not have enough digital visibility or marketing resources. Existing platforms usually promote only selected or partnered restaurants, which limits opportunities for other businesses to grow and attract customers.

Another important limitation in current systems is the lack of intelligent assistance. Users do not get personalized suggestions based on their preferences, cravings, or budget, which makes the selection process more difficult. Similarly, the absence of engaging features such as visual content reduces user interaction with the platform.

Therefore, there is a need for a system that can solve these problems by providing:

- A centralized platform for restaurant discovery
- Complete menu and price visibility
- Location-based restaurant suggestions
- AI-based personalized recommendations
- Equal digital exposure for all restaurants
- An engaging environment for users

The proposed system aims to address these issues by creating a unified solution that improves accessibility, transparency, and user experience while supporting restaurant growth in a competitive market.

2.2 Deliverables and Development Requirements

The project deliverables include the main software system, supporting documentation, and the required technical components needed to develop and run the application successfully. These

deliverables define what will be produced at the end of the project and what resources are required during development.

2.2.1 Software Deliverables

The main software deliverable is a mobile application that provides restaurant discovery, menu viewing, price comparison, location-based search, AI assistance, food reels, and online ordering features. The system will allow users to register, explore restaurants, view details, add items to cart, and place orders. Restaurants will be able to register their business, manage menus, upload offers, and promote their services.

The system will also include an admin module for managing users, restaurants, content, and platform activity. This will help maintain system control, verify restaurant data, and ensure smooth operation of the application

2.2.2 Documentation Deliverables

The project will include complete documentation required for final year project submission. These documents will explain the project idea, requirements, design, implementation, testing, and final outcome. The major documentation deliverables include:

- Scope Document
- Software Requirements Specification Document
- Software Design Description Document
- Final Project Report
- Testing Documentation
- User Manual or System Guide
- Presentation Slides

2.2.3 Development Requirements

The application requires a suitable development environment, frontend tools, backend services, database support, and APIs. The mobile application will be developed using Flutter, while Firebase services will be used for authentication, database management, storage, and real-time synchronization. Google Maps API will be used for location-based restaurant search and navigation support.

The AI assistant and recommendation feature will require logic for understanding user input and suggesting suitable restaurants or meals based on preferences, budget, and location. The

reels feature will require media storage and retrieval support so that promotional food videos can be uploaded and displayed within the application.

The basic development requirements include:

- Flutter SDK for mobile application development
- Dart programming language
- Firebase Authentication for user login and registration
- Firebase Firestore for database management
- Firebase Storage for images and videos
- Google Maps API for location services
- AI chatbot or recommendation logic
- Android Studio or Visual Studio Code as development tools
- Internet connection for cloud-based services
- Android device or emulator for testing

Overall, these deliverables and requirements provide the foundation for developing a functional, scalable, and user-friendly food discovery and ordering system.

2.3 Current System

In the current system, users mainly depend on food delivery applications, map services, social media pages, and personal recommendations to search for restaurants and food options. These platforms provide useful information, but they do not offer a complete and centralized solution for restaurant discovery, menu comparison, pricing, and personalized food selection.

Food delivery applications mainly focus on online ordering and delivery. They usually display only partnered restaurants, which limits the number of available choices for users. Map-based services help users find nearby restaurants and directions, but they do not provide complete menu details, proper price comparison, or direct food selection support. Social media pages show promotional content and reviews, but the information is scattered and not always reliable. Because of these limitations, users often need to switch between different platforms before making a decision. This process becomes time-consuming, especially when users want to compare restaurants according to price, location, rating, and food type. The problem becomes more noticeable for budget-conscious users and people visiting unfamiliar areas.

Table 2.1 Existing System Limitations

Existing System	Main Purpose	Limitation
Food Delivery Applications	Online food ordering and delivery	Limited to partnered restaurants and lacks complete comparison
Map-Based Services	Restaurant search and navigation	Does not provide full menus, prices, or ordering support
Social media Pages	Food promotion and reviews	Information is scattered and not standardized
Personal Recommendations	Informal restaurant suggestions	Limited, subjective, and not always reliable

Overall, the current systems provide partial support but do not fully solve the problem of restaurant discovery and informed decision-making. Therefore, a centralized platform is required where users can view restaurants, compare menus and prices, receive AI-based suggestions, and access location-based restaurant information within a single application.

Chapter 3

Requirement Analysis

3 Requirement Analysis

Requirement analysis defines the actual needs of the system before development starts. In this project, the requirements are identified by studying the problems faced by users, restaurant owners, and platform administrators. The system must provide a smooth way for users to discover restaurants, view menus, compare prices, place orders, and receive helpful suggestions according to their preferences. The application has different user roles, and each role requires separate functionality. Customers need easy access to restaurant information, menu details, location-based results, food reels, cart management, and AI-based recommendations. Restaurant owners need features for business registration, menu updates, offer management, promotional content, and order handling. The admin requires control over users, restaurants, content, and overall platform activity to ensure that the system remains reliable and organized. The requirements of the system are divided into two major categories: functional requirements and non-functional requirements. Functional requirements describe the main operations of the system, such as user registration, restaurant discovery, menu management, ordering, chatbot interaction, reels upload, loyalty points, and admin management. Non-functional requirements describe the quality of the system, including performance, usability, security, reliability, scalability, and maintainability. The system must be simple enough for normal users to operate without difficulty, but also strong enough to handle restaurant data, user activity, order records, and real-time updates. Since the application depends on online services, cloud storage, maps, and AI-based assistance, proper requirement analysis is necessary to ensure that all modules work together smoothly.

Overall, the requirement analysis helps define a clear foundation for the system by identifying what the users need, what the restaurants require, and what the platform must provide to operate successfully.

3.1 Use Case Diagram

The use case diagram represents the interaction between different actors and the system. It shows how customers, restaurant owners, content creators, AI assistant, and admin interact with the main features of the application. The diagram helps to understand the system boundaries and explains which functionality is available to each type of user.

In this system, the customer is the primary user who can register, log in, search restaurants, view menus, compare prices, place orders, use the AI assistant, watch reels, give ratings, and manage loyalty rewards. The restaurant owner interacts with the system by registering the restaurant, updating menu items, managing prices, uploading offers, responding to orders, and

promoting the restaurant through reels or featured deals. The admin has control over the overall platform. Admin can verify restaurant accounts, manage users, monitor uploaded content, remove inappropriate data, and maintain system activity. Content creators can upload food-related reels and promotional videos to increase engagement. The AI assistant acts as a supporting system component that provides recommendations based on user preferences, budget, location, and food cravings.

The main actors included in the use case diagram are:

- Customer
- Restaurant Owner
- Content Creator
- AI Assistant
- Administrator

The major use cases included in the system are:

- Register and login
- Search and browse restaurants
- View menus and prices
- Compare restaurants
- Place and manage orders
- Get AI-based recommendations
- Upload and view reels
- Manage restaurant profile and menu
- Manage offers and promotions
- Monitor users, restaurants, and content

The use case diagram provides a clear view of the system functionality and shows how each actor contributes to the overall workflow of the application

This diagram shows how the restaurant owner interacts with the system. It includes actions such as restaurant registration, menu management, price updates, offer posting, order handling, and promotional content management.

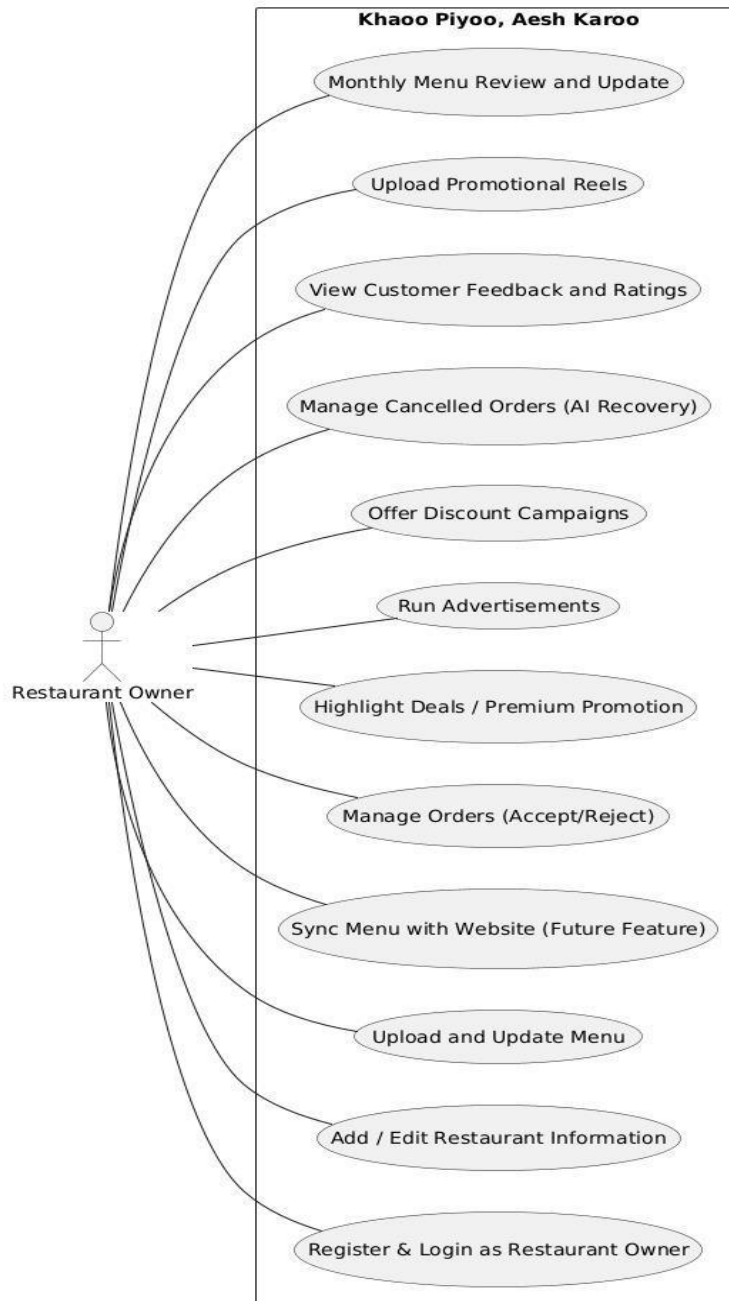


Figure 3.1 Use case diagram Restaurant owner

This diagram represents how customers interact with the system. It includes registration, login, restaurant browsing, menu viewing, price comparison, order placement, AI assistant usage, ratings, and loyalty rewards.

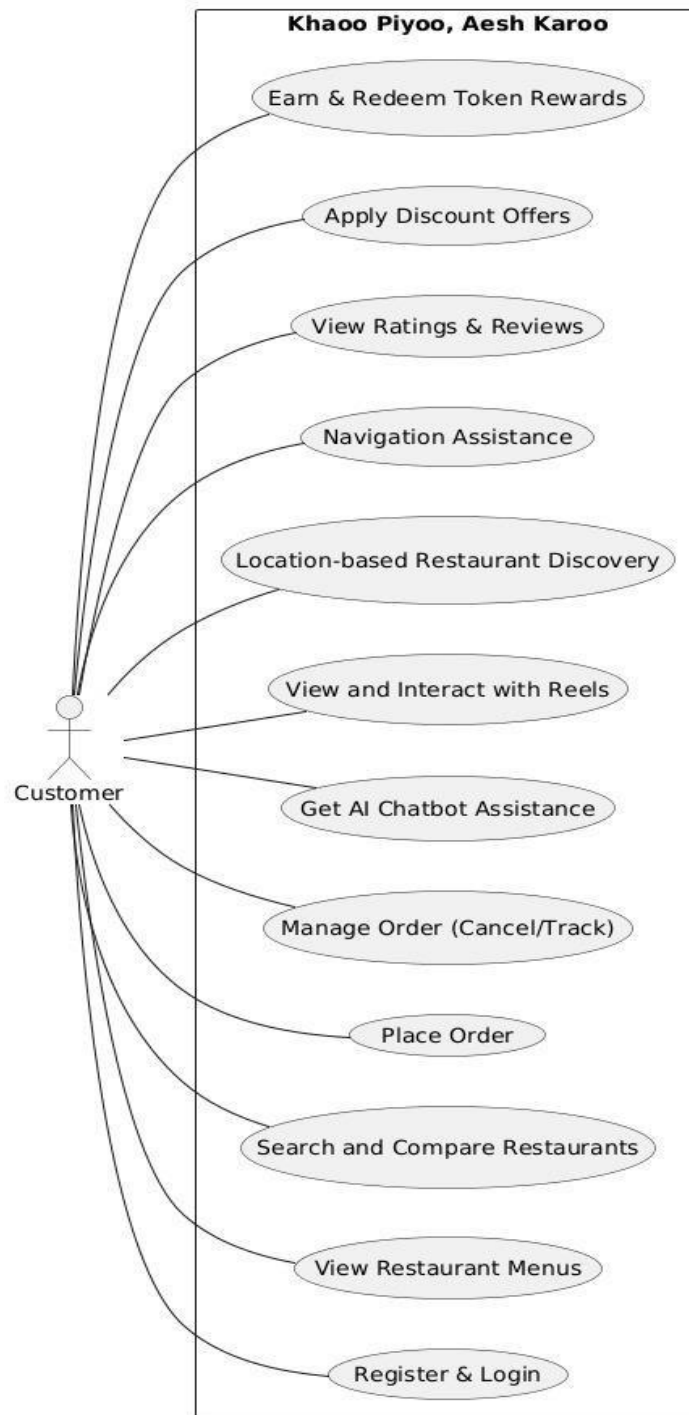


Figure 3.2 Use case diagram Customer Side

This diagram explains the role of content creators in the system. It includes uploading food reels, tagging restaurants, promoting food items, and increasing user engagement through visual content.

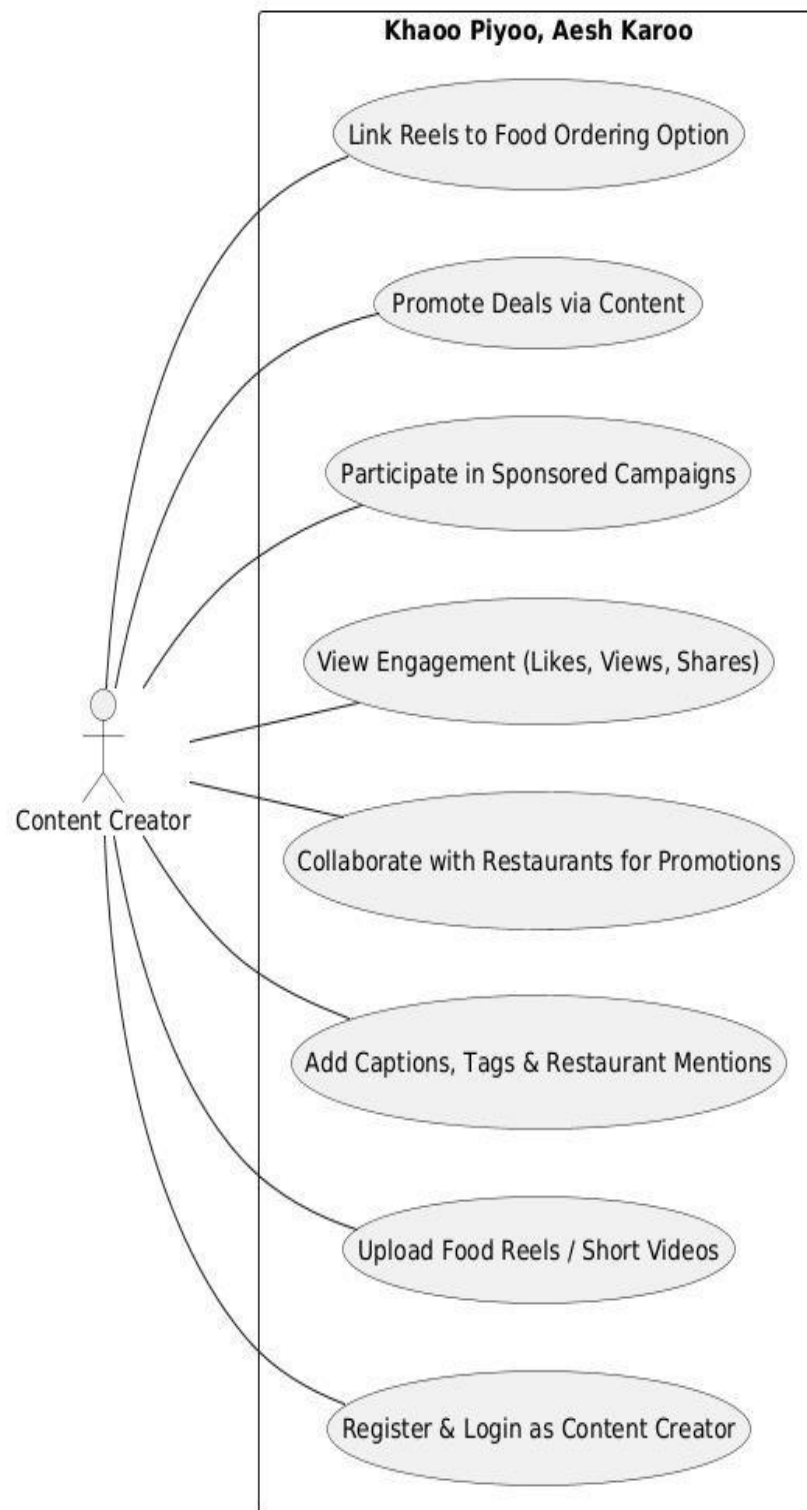


Figure 3.3 Use Case diagram Content creator

This diagram shows how the AI assistant supports users in selecting restaurants and meals. It uses user preferences, budget, location, and cravings to provide suitable recommendations.

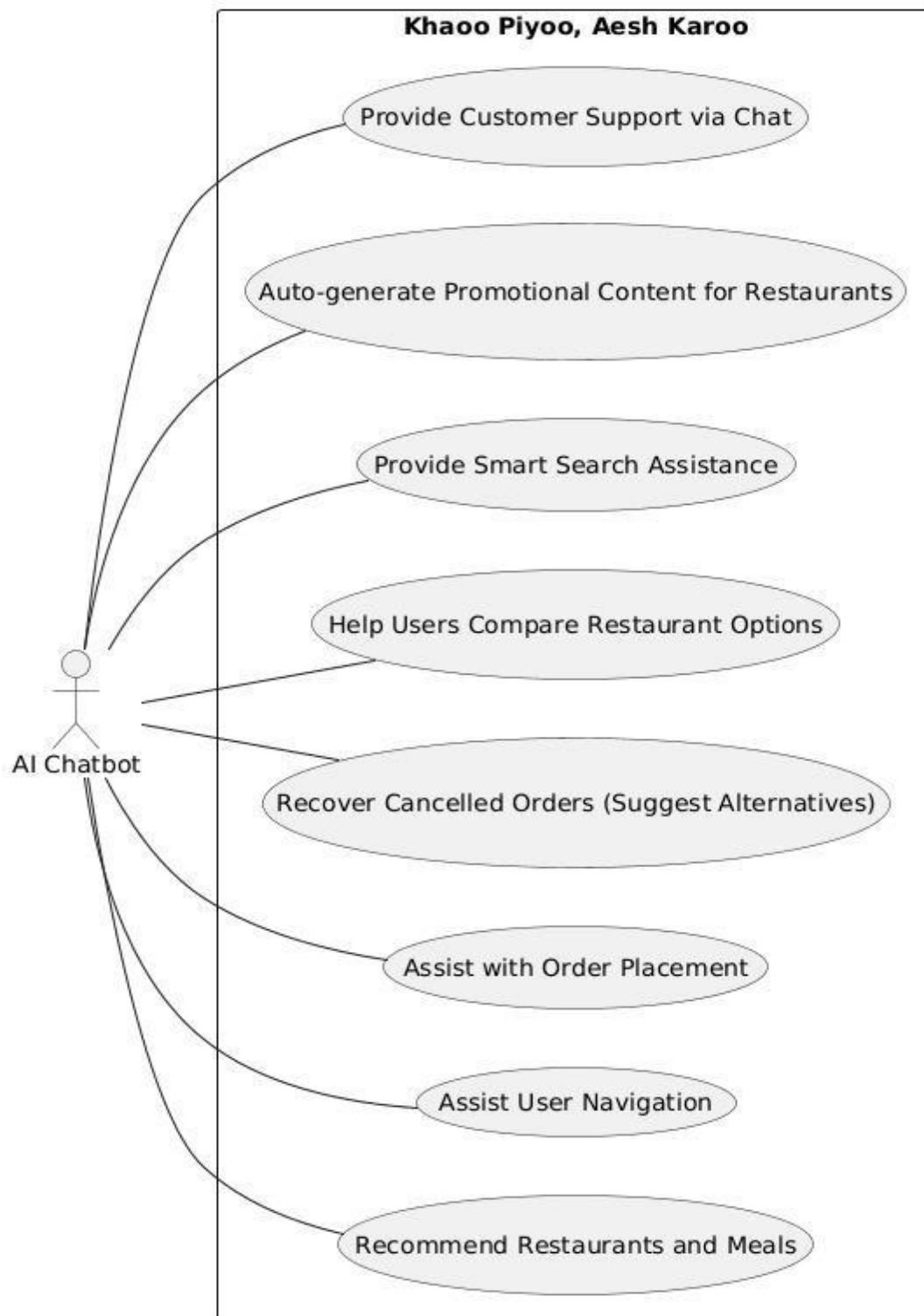


Figure3.4 Use Case Diagram Ai Chatbot

This diagram represents the administrator's role in managing the platform. It includes user management, restaurant verification, content monitoring, complaint handling, and maintaining overall system quality.

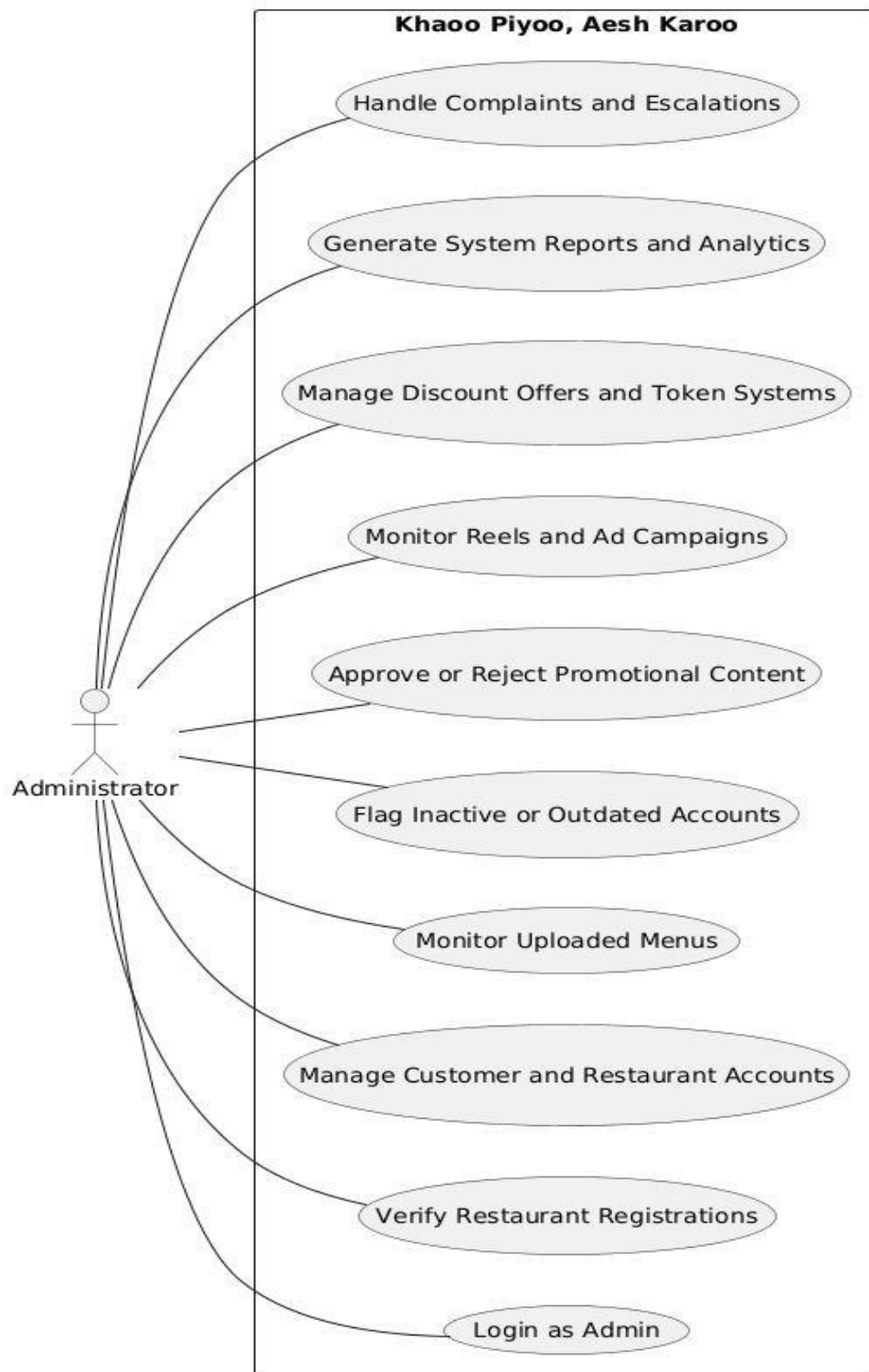


Figure 3.5 Use Case Diagram Admin Side

3.2 Detailed Use Case

Detailed use cases describe how different users interact with the system to perform specific tasks. They explain the purpose of each function, the actor involved, the required conditions, and the expected outcome after completion. These use cases help in understanding the behavior of the system from the user's point of view. The major use cases of the system include customer registration, restaurant discovery, menu viewing, order placement, AI-based recommendation, restaurant menu management, reels upload, and admin management. Each use case represents an important part of the application workflow and supports the overall goal of making restaurant discovery and ordering easier

3.2.1 Use Case Description

The Use Case Descriptions define how different actors interact with the system to achieve specific goals. Each use case outlines the flow of events, conditions, and expected outcomes to ensure clear understanding of system behavior.

Table 3.1 Browse Restaurants & Menus (Customer)

Use Case ID:	UC-1
Use Case Name:	Browse Restaurants & Menus
Actors:	Primary Actor: Customer Secondary Actors: Restaurant
Description:	A Customer opens the app to explore restaurants, their menus, and pricing details. The goal is to view available options, compare prices, and decide where to order from.
Trigger:	Customer selects the "Browse Restaurants" option from the home screen.
Preconditions:	PRE-1. Customer must have a registered account. PRE-2. Customer must be logged in.
Postconditions:	POST-1. List of restaurants and their menus is displayed. POST-2. Customer can select a restaurant to place an order or save it as favorite.
Normal Flow:	Customer selects "Browse Restaurants." System fetches the list of active restaurants. System displays restaurants with ratings, menu, and pricing. Customer filters or sorts based on price, location, or rating. Customer clicks on a restaurant to view full menu details. Customer either continues to order or exits.

Alternative Flows:	No restaurants found System displays “No restaurants available in your area.” Customer may adjust filters or retry.
Exceptions:	E1 — Network Error: System shows “Failed to load. Retry.” E2 — Server Timeout: Prompt retry or offline mode.
Business Rules:	BR-1 Only active restaurants are displayed. BR-2 Menus are sorted by restaurant rating by default.
Assumptions:	Customer has stable internet connection. Restaurants have uploaded their updated menus.

Table 3.2 Order Placing

Use Case ID:	UC-2
Use Case Name:	Place an Order
Actors:	Primary Actor: Customer Secondary Actors: Restaurant, Payment Gateway
Description:	A Customer selects items from a restaurant’s menu, adds them to the cart, and places an order.
Trigger:	Customer clicks “Place Order” in the cart.
Preconditions:	PRE-1. Customer must be logged in. PRE-2. Customer must have a valid payment method or COD option.
Postconditions:	POST-1. Order is saved with status “Pending.” POST-2. Restaurant is notified about the new order. POST-3. Inventory and delivery capacity are updated.
Normal Flow:	Customer selects menu items. System displays cart with total price. Customer confirms order. System sends order to restaurant. System generates order ID and confirmation message. Restaurant accepts the order.
Alternative Flows:	1.1 Payment Failure System notifies Customer. Customer retries payment or chooses COD.
Exceptions:	Insufficient inventory. System informs Customer and prompts modification.
Business Rules:	BR-1 Orders above PKR 1000 get free delivery. BR-2 Restaurant must accept or reject orders within 2 minutes.

Assumptions:	Payment gateway is online. Restaurant has real-time inventory update.
---------------------	--

Table 3.3 Manage Menu & Prices (Restaurant)

Use Case ID:	UC-3
Use Case Name:	Manage Menu & Prices
Actors:	Primary Actor: Restaurant Owner Secondary Actors: Admin
Description:	Restaurant owner adds, edits, or removes menu items and updates pricing, ensuring the menu is accurate for customers.
Trigger:	Restaurant selects “Menu Management” option.

Table 3.4 Manage Menu & Prices (Restaurant)

Use Case ID:	UC-3
Preconditions:	PRE-1. Restaurant must be registered and verified. PRE-2. Restaurant must be logged in.
Postconditions:	POST-1. Updated menu is saved and synced for customers. POST-2. AI bot receives updated data for recommendations.
Normal Flow:	Restaurant logs in. Selects “Menu Management.” Adds, edits, or deletes menu items. Saves changes. System updates database and displays changes on customer app.
Alternative Flows:	Invalid Price Input System shows validation error and requests correction.
Exceptions:	E1 Network Error — changes not saved. E2 Menu locked by Admin for review.
Business Rules:	BR-1 Prices must be in PKR. BR-2 All menu items require name, price, and category.
Assumptions:	Restaurant uses updated pricing regularly. Admin approves flagged menu updates.

Table 3.5 Upload Food Reels (Content Creator)

Use Case ID:	UC-4
Use Case Name:	Upload Food Reels

Actors:	Primary Actor: Content Creator Secondary Actors: Admin, Restaurant
Description:	Content Creator uploads short promotional reels to engage users and promote restaurants.
Trigger:	Creator selects “Upload Reel.”
Preconditions:	PRE-1. Creator must be logged in. PRE-2. Creator account must be approved.
Postconditions:	POST-1. Reel is uploaded and visible in the feed. POST-2. Restaurant tagged receives notification.
Normal Flow:	Creator logs in. Selects “Upload Reel.” Uploads video and adds caption, tags restaurant. System validates and saves reel. Reel becomes visible on the platform.

Table 3.6 Upload Food Reels (Content Creator)

Use Case ID:	UC-4
Alternative Flows:	1.1 Large File Error 1. System prompts to reduce size.
Exceptions:	1.0.E1 Inappropriate content flagged Admin removes or reviews it.
Business Rules:	BR-1 Max reel duration is 60 seconds. BR-2 Offensive content is auto-flagged.
Assumptions:	Creator follows community guidelines. Internet connection is stable.

Table 3.7 Recommend Meals & Restaurants (AI Bot)

Use Case ID:	UC-5
Use Case Name:	Recommend Meals & Restaurants
Actors:	Primary Actor: AI Bot Secondary Actors: Customer, Restaurant
Description:	AI bot analyzes user preferences, location, and budget to recommend restaurants and meals.
Trigger:	Customer interacts with the chatbot or enters a craving

Preconditions:	PRE-1. Customer must be logged in. PRE-2. Restaurants must have menu data.
Postconditions:	POST-1. AI Bot displays recommended restaurants/meals. POST-2. Customer can directly place an order.
Normal Flow:	Customer opens chatbot. Enters craving or budget. AI Bot fetches restaurant data. AI Bot filters and ranks options. AI Bot displays top recommendations.
Alternative Flows:	1.1 No match found 1. AI Bot suggests nearby popular meals.
Exceptions:	1.0.E1 Bot server timeout. System displays fallback “Browse manually.”
Business Rules:	BR-1 Recommendations are location-based. BR-2 AI Bot learns from previous choices.

3.3 Functional Requirements

Functional requirements define the main services and operations that the system must provide to its users. These requirements describe how the application behaves when customers, restaurant owners, content creators, and administrators interact with different modules of the system. The functional requirements of the system are divided according to the major modules of the application

3.3.1 User Registration and Authentication

The system shall allow users to create an account using required personal information such as name, email, phone number, and password. After registration, users shall be able to log in securely and access the features according to their role.

The system shall support different user roles, including customer, restaurant owner, content creator, and administrator. Each role shall have access only to its relevant dashboard and functions.

The system shall validate user credentials during login and display an appropriate message if the entered information is incorrect.

3.3.2 Restaurant Registration and Profile Management

The system shall allow restaurant owners to register their restaurant by providing business details such as restaurant name, location, contact information, food category, and description.

The system shall allow restaurant owners to update their restaurant profile whenever required. Updated information shall be reflected in the customer side of the application.

The system shall allow administrators to verify restaurant accounts before making them fully visible to users.

3.3.3 Menu Management

The system shall allow restaurant owners to add, update, and delete menu items. Each menu item shall include basic details such as item name, price, category, description, and availability status.

The system shall display updated menus to customers in real time so that users can view the latest prices and available food items.

The system shall prevent incomplete or invalid menu entries, such as missing item name or incorrect price format.

3.3.4 Restaurant Discovery and Search

The system shall allow customers to search restaurants by name, food type, category, location, and price range.

The system shall display restaurant details including menu, prices, ratings, location, offers, and contact information.

The system shall allow users to filter restaurants according to their preferences, such as nearby restaurants, affordable options, top-rated restaurants, or specific cuisine types.

3.3.5 Location-Based Restaurant Suggestions

The system shall detect the user's current location, after permission is granted, and display nearby restaurants accordingly.

The system shall provide navigation support so users can locate restaurants easily.

The system shall also allow users to manually select a location if they do not want to use current location access.

3.3.6 Online Ordering and Cart Management

The system shall allow customers to select food items from restaurant menus and add them to the cart. The system shall allow users to update item quantity, remove items, and review the total amount before confirming the order.

After order confirmation, the system shall generate an order record and notify the restaurant owner about the new order.

3.3.7 AI Chatbot and Recommendation System

The system shall provide an AI-based assistant that helps users choose restaurants and meals according to their cravings, budget, location, and preferences.

The chatbot shall accept user queries in simple language and return suitable suggestions.

The system shall recommend restaurants or food items based on available menu data, user preferences, and location information.

3.3.8 Food Reels and Content Engagement

The system shall allow restaurant owners and content creators to upload short food-related videos for promotion and engagement.

The system shall display uploaded reels in a scrollable feed where users can view food content.

The system shall allow users to interact with reels through basic engagement actions such as viewing, liking, and accessing the related restaurant profile.

3.3.9 Offers, Promotions, and Loyalty Rewards

The system shall allow restaurants to upload promotional offers, discounts, and highlighted deals.

The system shall display active offers to customers so they can discover discounted or promoted food items.

The system shall provide a loyalty reward feature where users can earn points or tokens based on their activity or orders. These tokens may be used for future discounts according to defined rules.

3.3.10 Admin Management

The system shall provide an admin dashboard for managing users, restaurants, content, and platform activity.

The admin shall be able to approve or reject restaurant registrations, monitor uploaded reels, manage reported content, and maintain overall platform quality.

The admin shall also be able to view basic system records such as registered users, active restaurants, menu updates, and order activity.

These functional requirements define the core behavior of the system and ensure that all major modules work together to provide a complete food discovery and ordering experience.

3.4 Non-Functional Requirements

Non-functional requirements define the quality and performance aspects of the system. These requirements ensure that the system not only performs its intended functions but also operates efficiently, securely, and reliably under different conditions.

3.4.1 Performance

The system should provide fast response times for user actions such as searching restaurants, viewing menus, and placing orders. Data should be loaded quickly to ensure a smooth user experience. The system should be able to handle multiple users at the same time without significant delay or system slowdown.

3.4.2 Usability

The application should have a simple and user-friendly interface so that users can easily navigate through different features. The design should be clean and intuitive, allowing even non-technical users to use the system without difficulty. Clear labels, buttons, and navigation flow should be maintained throughout the application.

3.4.3 Security

The system should ensure the protection of user data, including login credentials and personal information. Authentication mechanisms should be secure, and unauthorized access should be prevented. Data transmission between the user and the system should be protected to avoid misuse or data leakage.

3.4.4 Reliability

The system should function consistently without frequent crashes or failures. It should handle errors properly and provide appropriate messages when issues occur. The application should be available for use whenever required, ensuring dependable performance.

3.4.5 Scalability

The system should be able to handle growth in the number of users, restaurants, and data without affecting performance. As more users join and more restaurants are registered, the system should continue to operate efficiently.

3.4.6 Maintainability

The system should be designed in a way that allows easy updates, modifications, and improvements. Developers should be able to fix bugs, add new features, and update existing modules without affecting the overall system.

3.4.7 Compatibility

The application should work on different mobile devices and operating systems without major issues. It should support different screen sizes and device configurations to ensure accessibility for a wide range of users.

3.4.8 Availability

The system should remain available for users most of the time, especially during peak hours. Proper backend support and cloud services should ensure that the application remains active and responsive.

Chapter 4

Design and Architecture

4 Design and Architecture

The design and architecture of **Khaoo Piyoo** are structured to ensure scalability, modularity, and efficient performance. The system is divided into different components, where each component performs a specific function. This approach helps in maintaining a clear structure and allows smooth interaction between users and different parts of the application.

Khaoo Piyoo follows a layered architecture consisting of three main layers: presentation layer, application layer, and data layer. The presentation layer includes all user interfaces such as customer screens, restaurant dashboards, and admin panels. The application layer handles the core logic of the system, including restaurant discovery, order processing, AI-based recommendations, and content management. The data layer is responsible for storing and managing data such as user information, restaurant details, menus, orders, and media files.

To organize the system effectively, it is divided into different modules:

Customer Module: Allows users to search for restaurants, view menus, compare prices, place orders, and use the AI assistant.

Restaurant Module: Enables restaurant owners to register their business, manage menus, update prices, and handle customer orders.

AI Recommendation Module: Provides suggestions based on user preferences, budget, location, and food choices.

Reels and Content Module: Supports uploading and viewing of food-related videos for engagement and promotion.

Admin Module: Allows the administrator to manage users, restaurants, and overall system activity.

The architecture of **Khaoo Piyoo** is designed to support real-time data updates using cloud-based services. This ensures that any changes made in the system, such as menu updates or new offers, are immediately visible to users. The system also uses API-based communication between frontend and backend to ensure smooth data flow and processing.

Overall, the design of **Khaoo Piyoo** provides a flexible and maintainable structure where each module works independently while remaining connected within the system. This makes it easier to update features, improve performance, and expand the system in the future.

4.1 System Architecture

The system architecture of **Khaoo Piyoo** is based on a layered and modular structure. This architecture separates the user interface, application logic, and data management so that each

part of the system can work independently while remaining connected through proper communication. This makes the application easier to develop, test, maintain, and expand in the future.

The architecture consists of three main layers. The **presentation layer** includes the mobile application screens used by customers, restaurant owners, and administrators. Through this layer, users can search restaurants, view menus, place orders, upload content, and manage profiles. The **application layer** handles the main processing of the system, including authentication, restaurant discovery, menu management, order handling, AI recommendations, reels processing, and admin operations. The **data layer** stores all system information such as user records, restaurant details, menu items, orders, ratings, loyalty points, and uploaded media. The AI assistant is connected with user preferences, location information, and restaurant/menu data. It helps users by suggesting suitable meals or restaurants according to their needs. The reels module is connected with media storage, allowing food-related videos to be uploaded, stored, and displayed in the application.

The main architectural components are:

- Mobile application interface
- Authentication service
- Restaurant and menu management module
- Order management module
- AI recommendation module
- Reels and media storage module
- Admin management module

Overall, the architecture of **Khaoo Piyoo** supports smooth interaction between users, restaurants, and the system. It provides a scalable and maintainable foundation for current features as well as future improvements.[5]

4.2 Data Representation [Diagram + Description]

The data representation of **Khaoo Piyoo** explains how the main data entities of the system are organized and connected with each other. Since the system manages users, restaurants, menus, orders, reels, ratings, and loyalty rewards, a clear data structure is required to store and retrieve information efficiently.

The main entities of the system include **User, Restaurant, Menu Item, Order, Order Item, Rating, Reel, Promotion, and Loyalty Token**. A user can browse multiple restaurants, place orders, give ratings, and earn loyalty tokens. A restaurant can manage multiple menu items, upload promotional offers, receive orders, and publish reels. Each order contains one or more order items, and each order item is linked with a specific menu item.

The data relationships can be described as follows:

- One user can place many orders.
- One restaurant can have many menu items.
- One restaurant can receive many orders.
- One order can contain multiple order items.
- One user can give ratings to restaurants.
- One restaurant can upload many reels and promotions.
- One user can earn and redeem loyalty tokens.

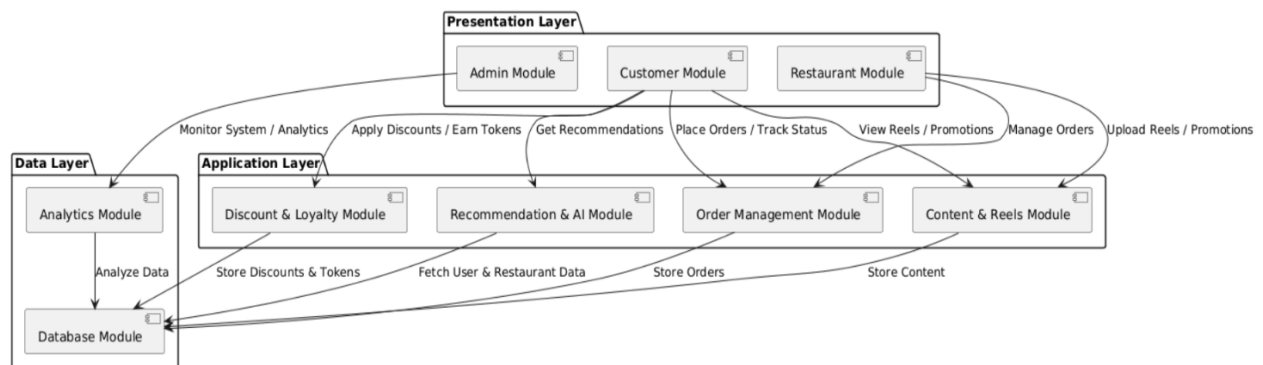


Figure 4.1 Data Representation Diagram

4.3 Process Flow/Representation

The process flow of **Khaoo Piyoo** describes how users interact with the system from the beginning of the application usage until the completion of major tasks such as restaurant discovery, AI-based recommendation, menu viewing, and order placement. The flow is designed to be simple and user-friendly so that users can move from one feature to another without confusion.

The process starts when a user opens the application and registers or logs in to the system. After successful authentication, the user is directed to the home screen where different options are available, such as browsing restaurants, searching food items, viewing reels, checking offers, or interacting with the AI assistant. If the user wants to search manually, the system

displays restaurants based on location, category, ratings, or price range. The user can then view restaurant details, check menu items, compare prices, and select food according to preference. If the user chooses to use the AI assistant, the system collects input such as craving, budget, location, or food type. Based on this input, the AI assistant suggests suitable restaurants or meals. The user can then open the suggested restaurant, view the menu, add items to the cart, and place an order. Once the order is confirmed, the restaurant receives the order request and updates the order status accordingly.

For restaurant owners, the process begins with registration and verification. After approval, they can manage their profile, upload menus, update prices, add offers, and upload reels for promotion. The admin monitors these activities, verifies restaurants, manages content, and ensures that the platform remains organized and reliable.

- User opens the application
- User registers or logs in
- System verifies credentials
- User accesses the home screen
- User searches restaurants manually or uses AI assistant
- System displays restaurant and menu options
- User selects food items and adds them to cart
- User confirms the order
- Restaurant receives and processes the order
- System updates order status
- Admin monitors users, restaurants, and content

The process flow diagram explains the overall working of the system from a user's perspective. It shows the sequence of steps starting from application launch, user authentication, restaurant discovery, menu selection, cart management, and order confirmation. This diagram helps in visualizing how different modules are connected and how users navigate through the system. It ensures that all steps follow a logical sequence and that no critical process is missed. It also supports the identification of bottlenecks and improves the overall efficiency of the application.

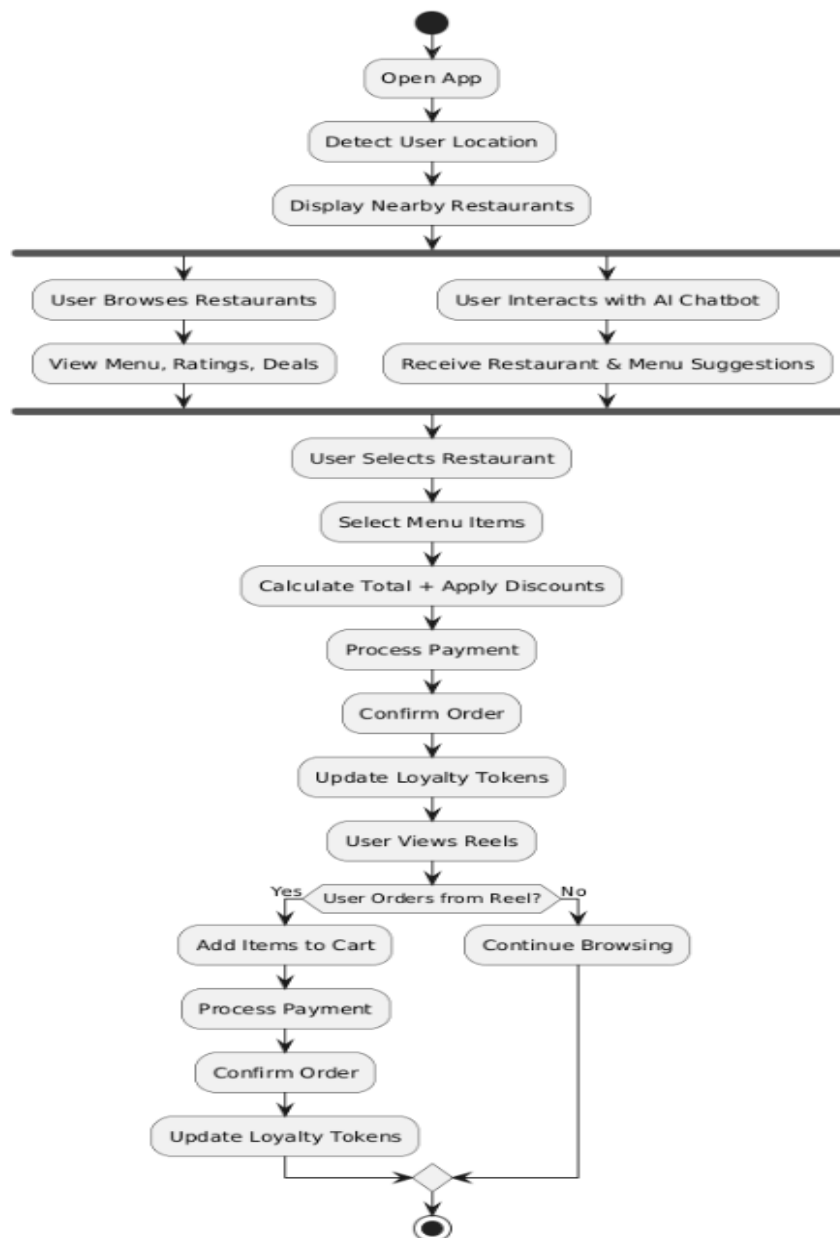


Figure 4.2 Process Flow Diagram

4.4 Design Models

Design models represent the internal structure and behavior of **Khaoo Piyoo**. These models help explain how different components of the system interact with each other and how data moves between users, restaurants, and the application. They also provide a clear understanding of the system before implementation, making development more organized and easier to manage.

The main design models used in this project include the class diagram, sequence diagram, state transition diagram, and data flow diagram. Each model explains a different aspect of the system. The class diagram shows the main entities and their relationships, while the sequence diagram explains the step-by-step interaction between the user and the system. The state transition diagram shows how the system changes from one state to another during order placement, and the data flow diagram explains how information moves between different modules

4.4.1 Class Diagram

The class diagram includes major classes such as *User*, *Customer*, *Restaurant*, *MenuItem*, *Order*, *OrderItem*, *Rating*, *Reel*, *LoyaltyToken*, *AI Assistant*, and *Admin*. These classes represent the main objects of the system and define their attributes and functions. For example, the *Restaurant* class manages restaurant details and menu items, while the *Order* class stores order information and status.

- **Aggregation:** A *Restaurant* **aggregates** multiple *Menu* items.
- **Composition:** An *Order* **composes** *OrderItem* objects; if the *Order* is deleted, associated items are also deleted.
- **Inheritance/Generalization:** *Customer* and *Admin* inherit from *User*.

The class diagram models the static structure of **Khaoo Piyoo** by identifying the main classes, their attributes, operations, and relationships. Core classes include *User*, *Customer*, *Restaurant*, *MenuItem*, *Order*, *OrderItem*, *Rating*, *Reel*, *LoyaltyToken*, *AI Assistant*, and *Admin*. Each class encapsulates relevant data and behavior; for instance, the *Restaurant* class maintains details such as name, location, and menu, while the *Order* class manages order ID, status, and total amount. The diagram clarifies how responsibilities are distributed and how data is organized across the system.

Relationships among classes are explicitly defined to ensure consistency and maintainability. Aggregation is used where a *Restaurant* contains multiple *MenuItem* objects, indicating a “has-a” relationship without strict ownership. Composition is applied in the *Order–OrderItem* relationship, where *OrderItem* objects depend on the lifecycle of an *Order*. Generalization (inheritance) is used to model common behavior, where *Customer* and *Admin* inherit shared properties from *User*. These relationships improve reusability, reduce redundancy, and support future extensions.

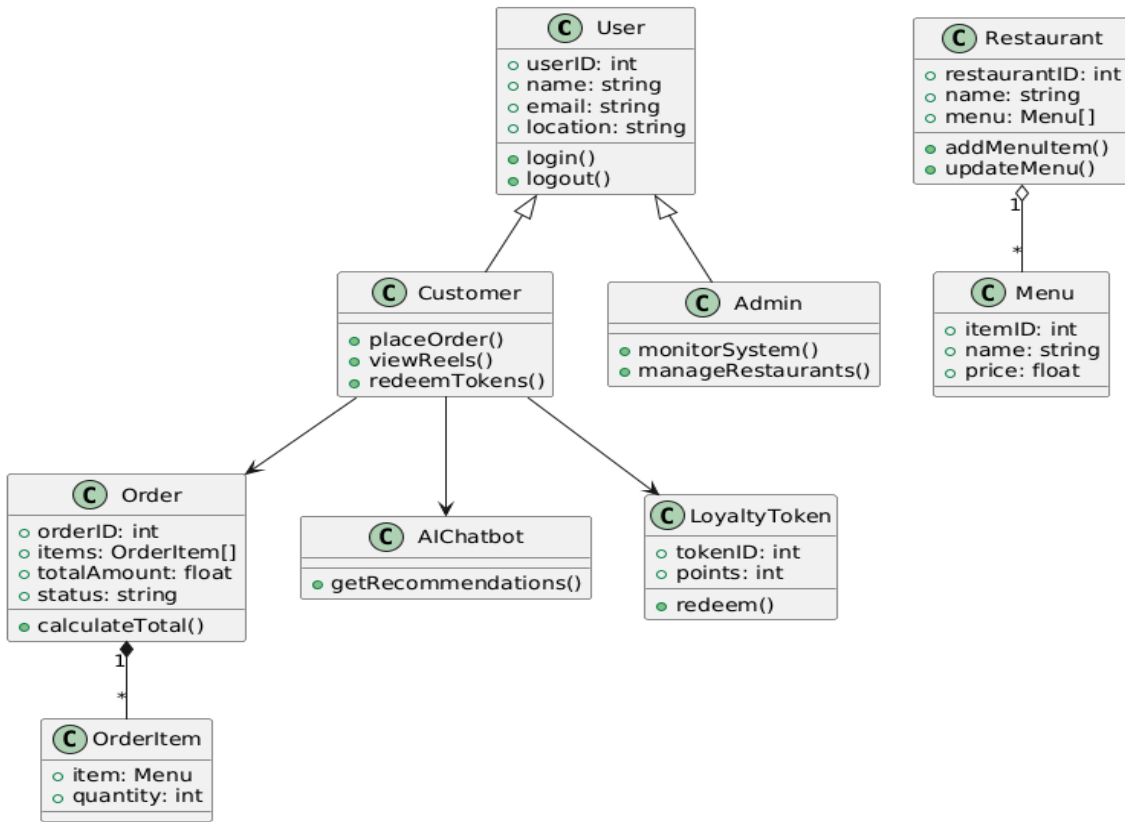


Figure 4.3 Class Diagram

4.4.2 Sequence Diagram

The sequence diagram represents important system interactions such as login, restaurant browsing, AI recommendation, and order placement. It shows how the customer sends a request, how the system processes it, how data is fetched from the database, and how the final response is displayed to the user.

The sequence diagram illustrates the dynamic interaction between system components during key operations such as login, restaurant browsing, AI recommendations, and order placement. It shows the chronological order of messages exchanged among the *User Interface*, *Application Logic*, *Database*, and supporting services. Each interaction is represented as a message flow, making it clear how a user request is processed from initiation to response.

For example, during order placement, the user selects items, the system validates the cart, communicates with the database to store order details, and returns a confirmation to the user. The diagram highlights synchronization points, request–response cycles, and dependencies between components, ensuring that all processes follow a consistent and predictable flow.

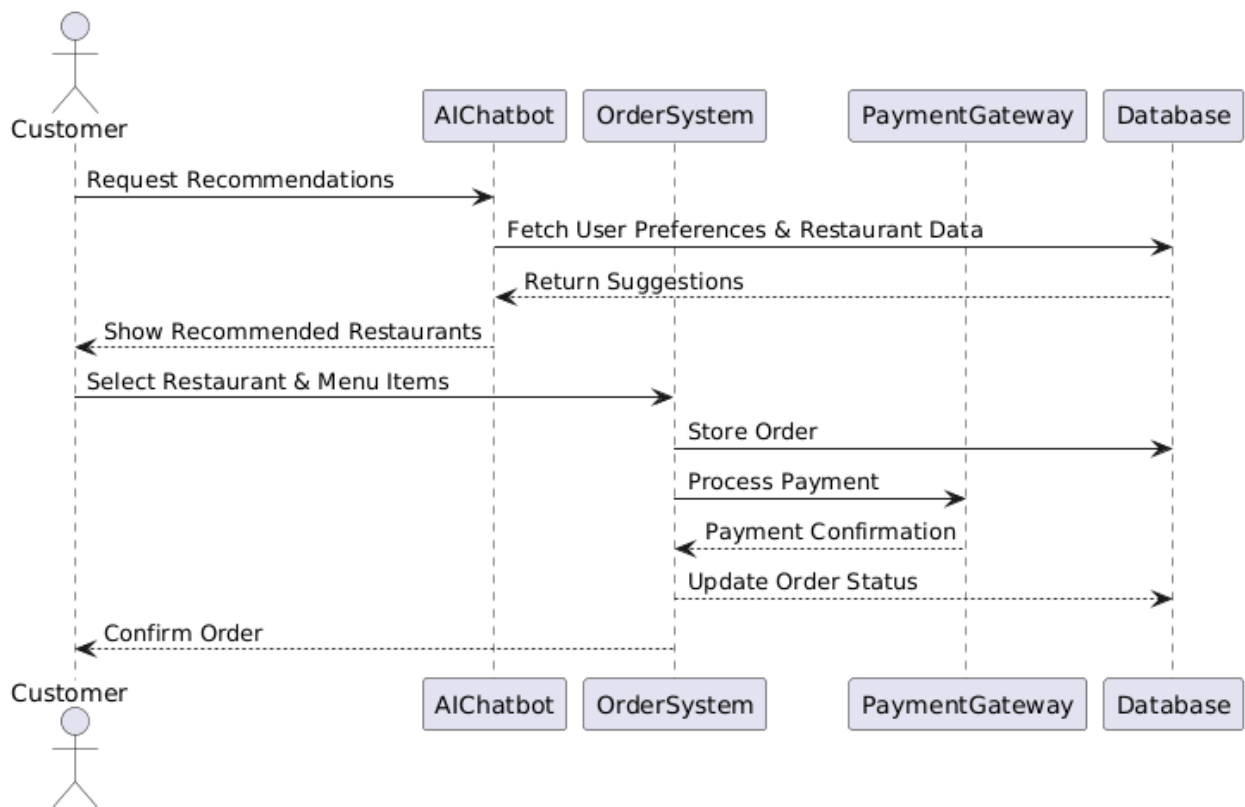


Figure 4.4 Sequence Diagram

4.4.3 State Transition Diagram

The State Transition Diagram shows how a customer’s journey flows within the “Khao Piyoo” food-ordering system, starting from browsing delicious items to placing and receiving their order. It highlights how the system moves between states such as menu selection, order confirmation, payment processing, and delivery tracking. Each transition reflects real user interactions, ensuring a smooth, enjoyable, and hassle-free food experience. The diagram captures how the system responds to actions like updating the cart, modifying choices, or completing payment, keeping the entire process seamless and user-friendly.

The state transition diagram represents the lifecycle of an order within the system. It shows how an order moves through different states such as *Created*, *Pending*, *Accepted*, *In Progress*, *Completed*, and *Cancelled*. Each state change is triggered by specific events, such as user confirmation, restaurant acceptance, or cancellation.

This model ensures that the system handles order processing in a controlled and consistent manner. It also helps in identifying invalid transitions and defining clear rules for status

updates. By formalizing state changes, the system reduces ambiguity and improves reliability in handling real-time order operations.

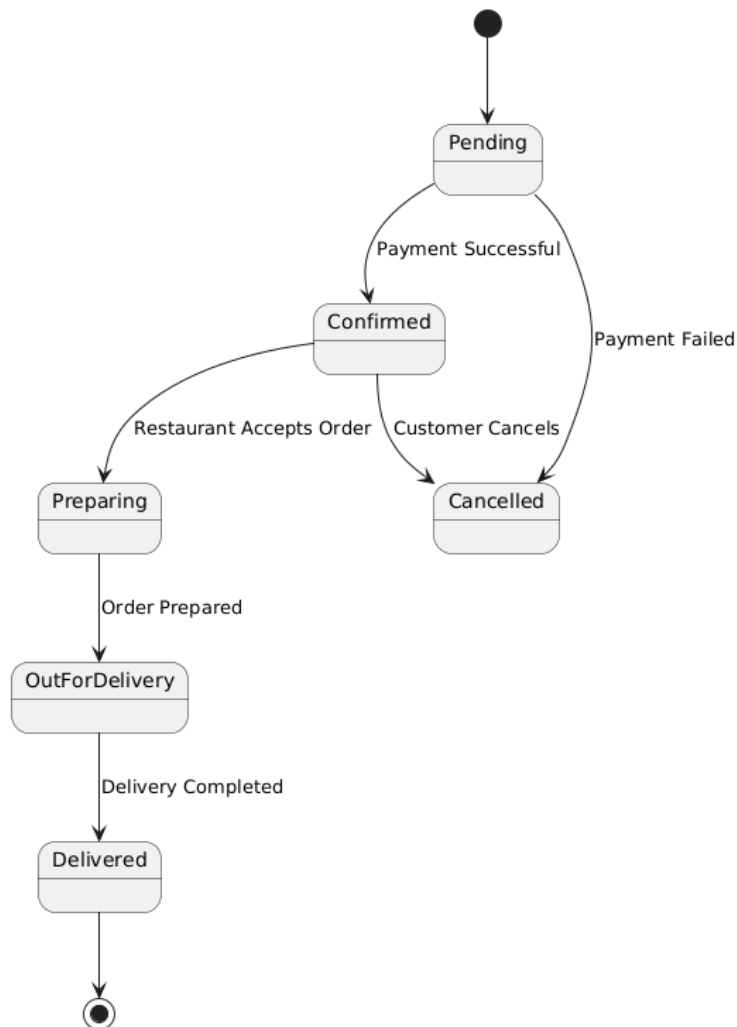


Figure 4.5 State Transition Diagram

4.4.4 Data Flow Diagram

The Data Flow Diagram illustrates how user requests like browsing menus, placing orders, and making payments flow through the Universal Food App system. It shows how data moves between users, the ordering module, the restaurant database, and delivery services to ensure a smooth food-ordering experience. This diagram highlights the system’s ability to process customer actions quickly and accurately for a seamless “Khaoo Piyoo” journey.

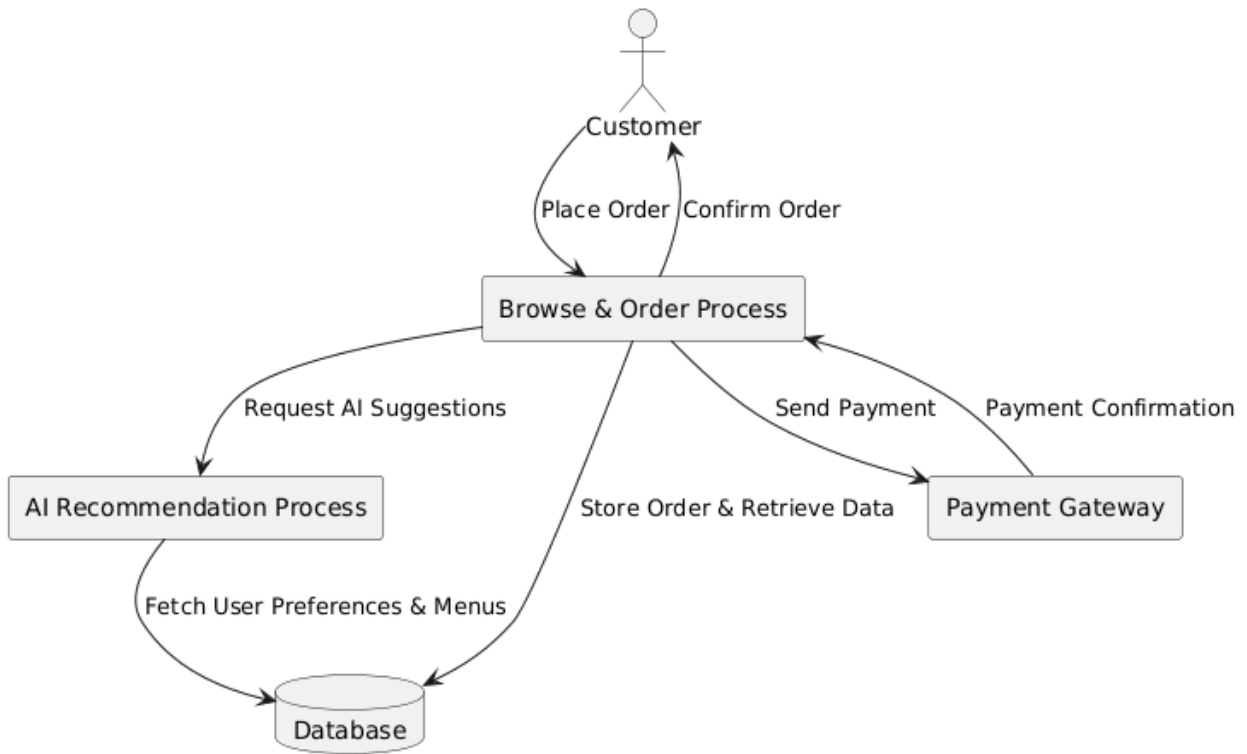


Figure 4.6 Data Flow Diagram

The data flow diagram (DFD) describes how information moves between different parts of **Khaoo Piyoo**. It shows the flow of data between external entities (customers, restaurant owners, admin), internal processes (authentication, restaurant management, order processing, AI recommendation), and data stores (database, media storage).

The diagram helps in understanding how user inputs are transformed into system outputs. For example, a user's search query is processed, matched with stored restaurant data, and returned as a filtered result. Similarly, order data flows from the user interface to the database and then to the restaurant module. This representation ensures transparency in data handling and supports efficient system design.

Chapter 5

Implementation

5 Implementation

The implementation of **Khaoo Piyoo** focuses on converting the system design into a working application. In this phase, the planned modules, interfaces, database structure, and system processes are developed and connected with each other. The implementation does not include source code in the report, but it explains how the major features are built and how different technologies are used to support the system.

The application is implemented as a mobile-based platform supported by backend services and cloud storage. The frontend provides screens for customers, restaurant owners, and administrators, while the backend manages authentication, restaurant data, menus, orders, reels, and user activity. The system also integrates AI-based recommendation logic to assist users in selecting restaurants and meals according to their preferences, budget, and location.

The main implementation areas of the system include:

- User authentication and profile management
- Restaurant registration and menu management
- Restaurant search and location-based discovery
- Cart and order processing
- AI assistant and recommendation handling
- Reels and promotional content management
- Loyalty rewards and offers
- Admin monitoring and control

The implementation follows the modular design explained in the previous chapter. Each module is developed separately and then integrated into the complete system. This approach helps reduce complexity and makes testing easier. For example, authentication can be developed and tested first, followed by restaurant listing, menu management, ordering, and AI assistance.

Cloud-based services are used to store and manage data in real time. When a restaurant owner updates a menu item, the change is saved in the database and becomes visible to customers. Similarly, when a customer places an order, the system records the order details and sends them to the restaurant side for further action.

Overall, the implementation phase ensures that the system features work together as planned and provide a smooth experience for users, restaurants, and administrators.[1] [4]

5.1 Algorithm

The algorithms used in **Khaoo Piyoo** define the internal working of the major system modules. These algorithms help explain how the system receives input, processes data, applies conditions, communicates with the database, and displays output to the user. Instead of adding source code, this section describes the logic of the main features in pseudocode and natural language form.

The major algorithms are related to user authentication, restaurant discovery, menu and cart management, order placement, AI-based recommendations, reels display, loyalty tokens, and admin management. Each algorithm supports a specific module and ensures that the system performs its functions in an organized way.

5.1.1 User Registration and Login Algorithm

This algorithm manages account creation and secure access to the application. When a user opens the application, the system allows them to either create a new account or log in to an existing account. During registration, the system checks whether all required fields are complete and valid. During login, the entered credentials are verified before granting access

Pseudocode

Start

User opens the application

User selects Register or Login

If Register is selected:

 Enter name, email, phone number, and password

 Validate all required fields

 If information is valid:

 Create user account

 Store user record in database

 Redirect user to dashboard

 Else:

 Display validation error

If Login is selected:

 Enter email or phone number and password

 Verify credentials

 If credentials are correct:


```
        Redirect user to dashboard
    Else:
        Display login error message
End
```

5.1.2 Restaurant Discovery Algorithm

This algorithm helps customers find restaurants according to their location, selected category, rating, price range, or search keyword. The system retrieves restaurant data from the database and displays the most relevant options to the user. If the user allows location access, nearby restaurants are shown first. Otherwise, the user can search manually.

Pseudocode

```
Start
User opens restaurant discovery screen
System checks location permission
If location permission is granted:
    Fetch user's current location
    Retrieve nearby restaurants from database
Else:
    Ask user to enter location manually
If user applies search or filter:
    Match restaurants according to selected criteria
    Display filtered restaurant list
Else:
    Display available restaurants by default
User selects a restaurant
System displays restaurant details, menu, prices, ratings, and offers

End
```

5.1.3 Menu and Cart Management Algorithm

This algorithm handles the process of selecting food items and managing the cart before placing an order. The user can add items, increase or decrease quantity, remove items, and review the total amount. The system ensures that unavailable items are not added to the cart.

Pseudocode

```
Start
```

User opens restaurant menu

User selects food item

If item is available:

 Add item to cart

 Update quantity if required

 Calculate subtotal

Else:

 Display item unavailable message

User reviews cart

System calculates total amount

If user updates cart:

 Recalculate total amount

If user confirms cart:

 Proceed to order placement

Else:

 Return to menu screen

End

5.1.4 Order Placement Algorithm

This algorithm processes the order after the user confirms the cart. It verifies item availability, calculates the final amount, creates an order record, and sends the order request to the restaurant. The order status is updated during the process so that both user and restaurant can track progress.

Pseudocode

Start

User confirms cart

System validates selected items and total amount

If all items are available:

 Generate order ID

 Save order details in database

 Set order status to Pending

 Notify restaurant owner

 Display order confirmation to user

Else:

 Display unavailable item message

 Ask user to modify cart

Restaurant accepts or rejects order

If restaurant accepts:

 Update order status to Accepted

Else:

 Update order status to Rejected

 Notify user

End

5.1.5 AI Recommendation Algorithm

This algorithm is used by the AI assistant to suggest suitable restaurants or meals. The system analyzes the user's input, such as craving, budget, cuisine preference, and location. It then filters available restaurants and menu items according to those conditions and displays the best matching results.

Pseudocode

Start

User opens AI assistant

User enters query such as craving, budget, or food preference

System analyzes user input

Fetch restaurant and menu data from database

Filter results based on:

- User location
- Food category
- Budget range
- Availability
- Ratings

Rank matching restaurants or meals

Display recommended options to user

If no exact match is found:

- Display nearby popular alternatives

End

5.1.6 Reels Display Algorithm

This algorithm controls how food-related reels are displayed to users. The system fetches approved reels from storage and displays them in a scrollable feed. Users can view reels, like them, and visit the related restaurant profile directly from the reel.

Pseudocode

Start

User opens reels section

System fetches active reels from database/storage

Sort reels based on latest uploads or engagement

Display reels in scrollable feed

User views, likes, or opens related restaurant profile

If user selects restaurant from reel:

- Redirect to restaurant detail page

End

5.1.7 Loyalty Token Algorithm

This algorithm manages reward points earned by users after successful orders. When an order is completed, the system calculates tokens according to the order value and adds them to the user's account. These tokens can later be redeemed for discounts based on platform rules.

Pseudocode

Start
Order status changes to Completed
System checks order amount
Calculate loyalty tokens based on order value
Add tokens to user's account
Update loyalty balance
If user applies tokens on next order:
 Validate available tokens
 Apply discount according to rules
 Deduct used tokens from balance

End

5.1.8 Admin Management Algorithm

This algorithm explains how the administrator manages the platform. The admin can verify restaurants, monitor users, review uploaded reels, manage reports, and take action against inappropriate activity. This helps maintain system quality and security.

Pseudocode

Start
Admin logs in to dashboard
System verifies admin credentials
Admin selects management option:
 Manage users
 Verify restaurants
 Review reels
 Monitor orders
 Manage reports or complaints
If action is valid:
 Update system records
 Display success message
Else:
 Display error message
End

Table 5.1 Details of APIs Used in the Project

Name of API	Description of API	Function/Class Name in Which It Is Used
Firebase Authentication API	Provides secure user authentication services	AuthService, LoginController, RegisterController
Firebase Firestore API	Cloud-based NoSQL database service	DatabaseService, RestaurantService, OrderService
Firebase Storage API	Cloud storage for media files	StorageService, ReelUploadController
AI Recommendation API	Provides intelligent suggestions	AIService, ChatbotController
REST API	Enables communication between frontend and backend	ApiService, Repository Classes

5.2 External APIs

External APIs are used in **Khao Piyoo** to provide essential services such as authentication, data storage, media handling, location tracking, notifications, and intelligent recommendations. These APIs reduce development complexity and allow the system to use reliable and scalable third-party services.

This table presents the external APIs integrated into the system along with their functionality and usage. It shows how each API contributes to different modules such as authentication, database management, location services, media storage, notifications, and AI-based recommendations. The table also identifies the main functions or classes where these APIs are used, providing a clear understanding of their role in system implementation.

5.3 User Interface

The user interface of **Khao Piyoo** is designed to provide a simple, attractive, and user-friendly experience. The application uses a modern design with consistent colors, icons, and layouts to ensure smooth navigation for users. The interface focuses on accessibility and ease of use so that users can quickly perform actions such as browsing restaurants, placing orders, and interacting with the AI assistant.

5.3.1 Landing Page / Website Interface

This Figure shows the landing page of **Khaoo Piyoo**, which introduces the platform to users. It highlights the main features such as AI-powered food discovery, restaurant comparison, and ordering functionality. The interface includes navigation options, call-to-action buttons like “Explore Now” and “Try AI Assistant,” and a preview of the mobile application. This screen is designed to attract users and provide a clear overview of the system.

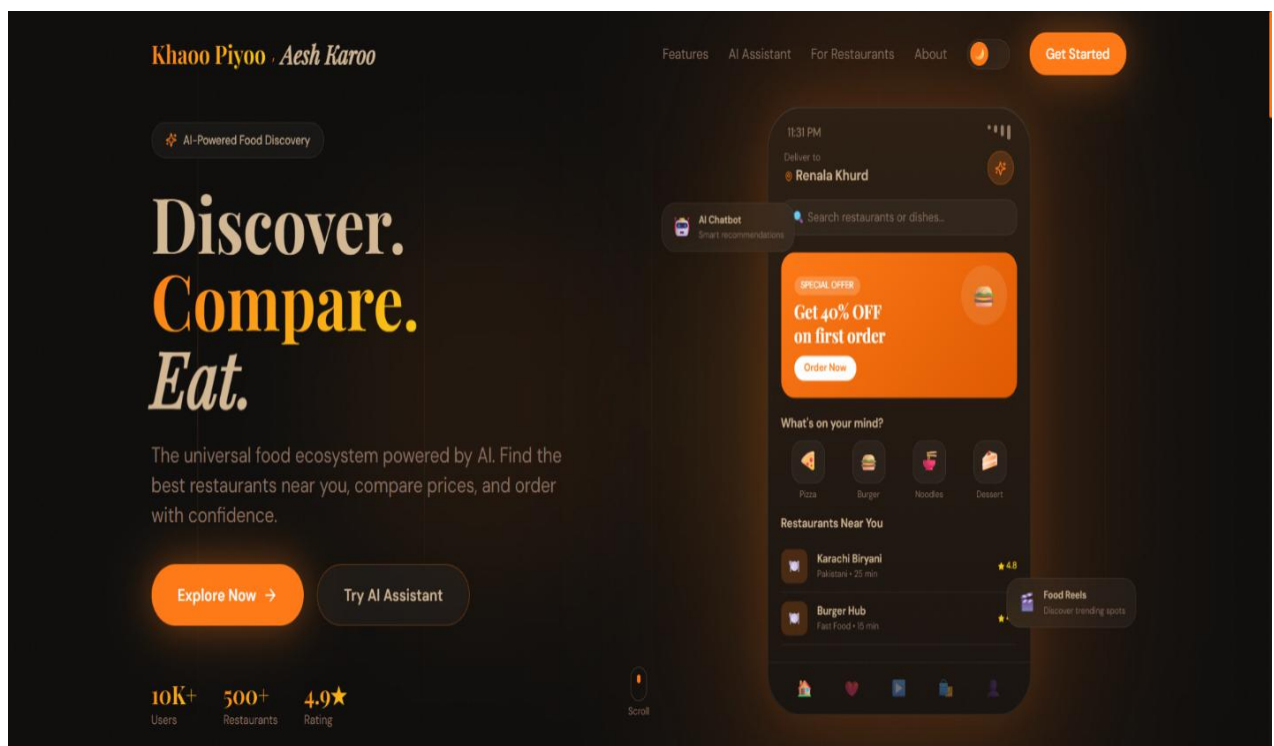


Figure 5.1 Landing Page / Website Interface

Light Mode

Light mode provides superior contrast for long-form text, making it the standard for reading-heavy sites. It is widely used to prioritize high readability and clarity, particularly in bright environments. Light mode provides clear visibility and strong readability for users, especially when reading longer text or browsing detailed information. It is commonly used in applications where clarity, simplicity, and ease of navigation are important. In **Khaoo Piyoo**, light mode helps users view restaurant details, menus, prices, buttons, and input fields more comfortably, particularly in bright environments. It also supports a clean and professional interface, making the application easier to understand and use for all types of users.

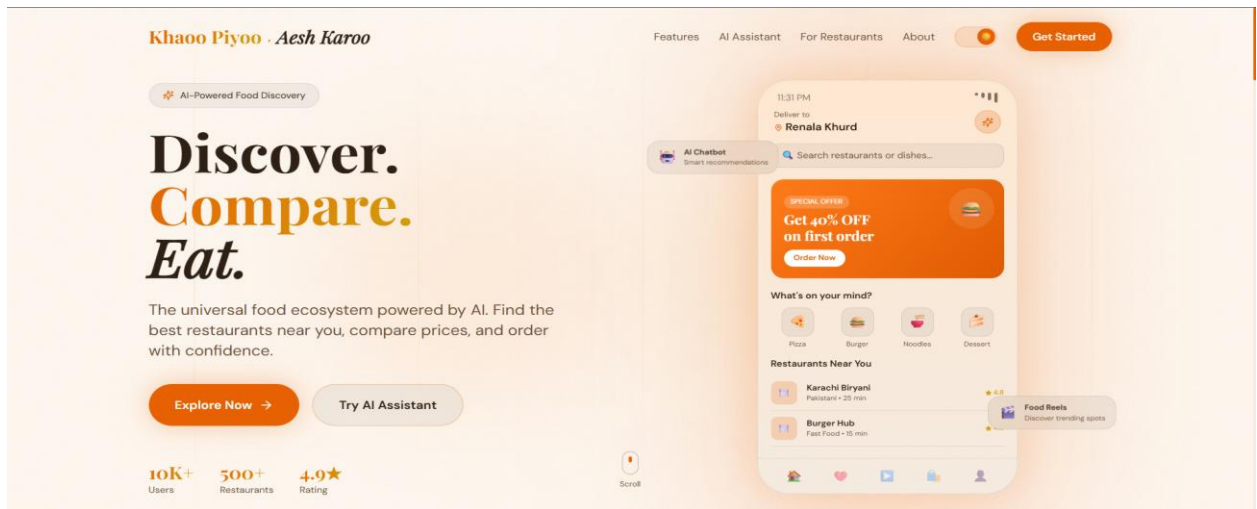


Figure 5.2 Landing Page / Website Interface

5.3.2 Role Selection Screen

This Figure shows the role selection interface of Khao Piyoo, where users can choose whether to continue as a customer or a restaurant owner. This allows the system to provide a customized experience based on the selected role.

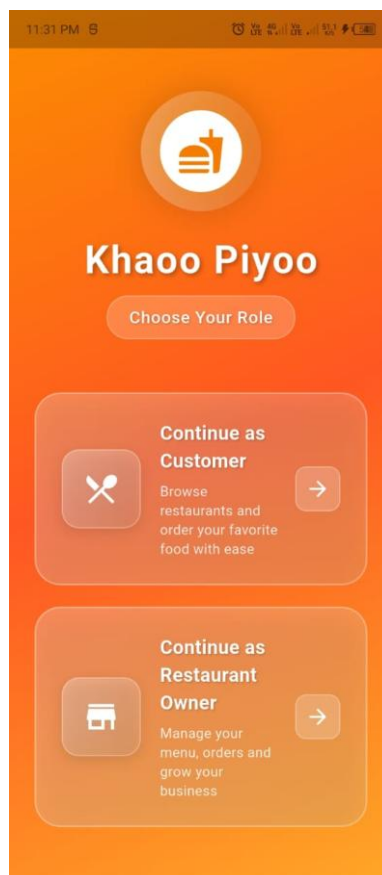


Figure 5.3 Role Selection

5.3.3 User Profile Screen

This Figure shows the AI assistant feature of **Khaoo Piyoo**. This interface allows users to interact with the system using simple text queries. The assistant provides intelligent recommendations based on user preferences, such as suggesting the best restaurants or meals. It improves user experience by making decision-making easier and faster.

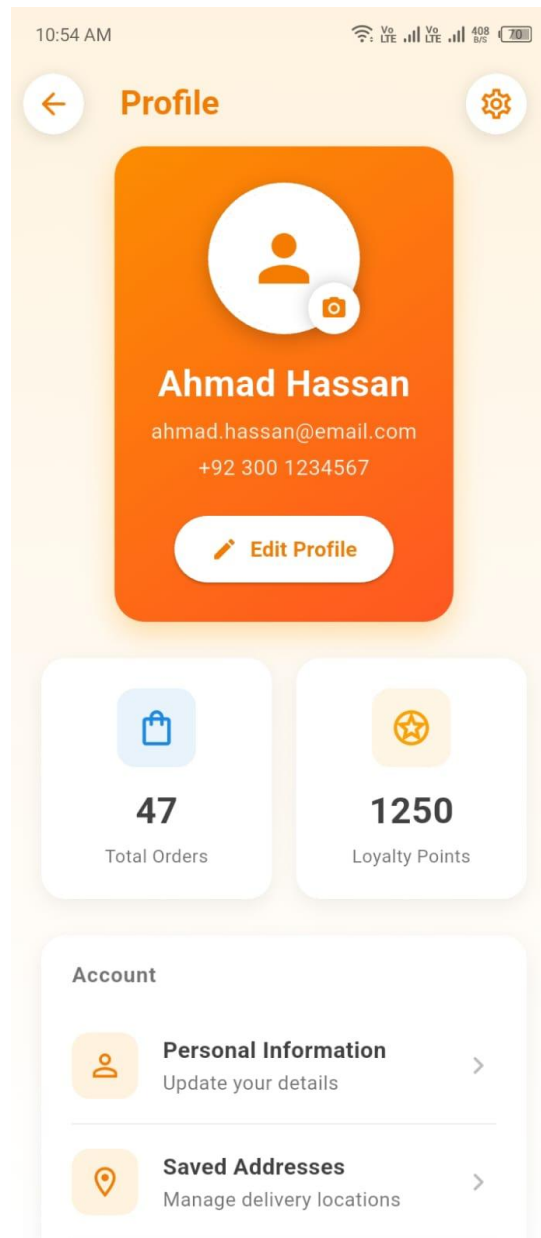


Figure 5.4 User Profile Screen

5.3.4 Reels / Content Screen

This Figure shows the reels section of the application, where users can view food-related videos. This feature provides an engaging experience by allowing users to explore restaurants visually. Users can interact with reels and directly access restaurant details from the content.

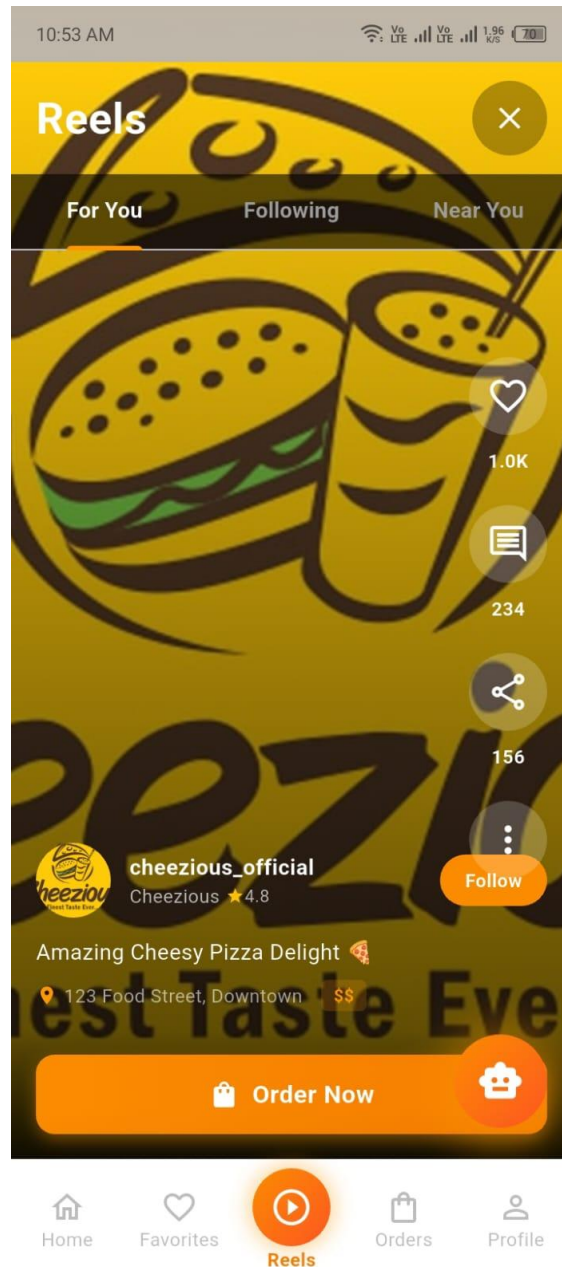


Figure 5.5 Reels / Content Screen

5.3.5 AI Assistant Interface

This Figure shows the AI assistant feature of **Khao Piyoo**. This interface allows users to interact with the system using simple text queries. The assistant provides intelligent recommendations based on user preferences, such as suggesting the best restaurants or meals. It improves user experience by making decision-making easier and faster

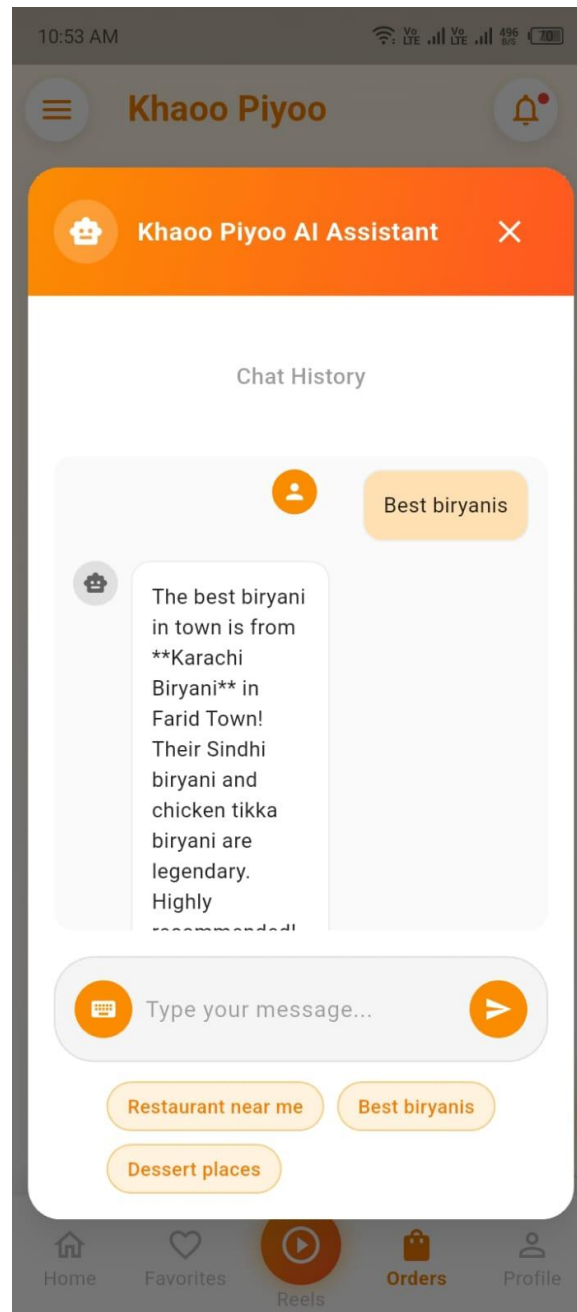


Figure 5.6 AI Assistant Interface

5.3.6 Cart Screen

This Figure displays the cart interface of the system. It shows selected food items, their prices, quantity controls, and total bill calculation. Users can update item quantities, apply promo codes, and proceed to checkout. This screen ensures that users can review their order before confirming it

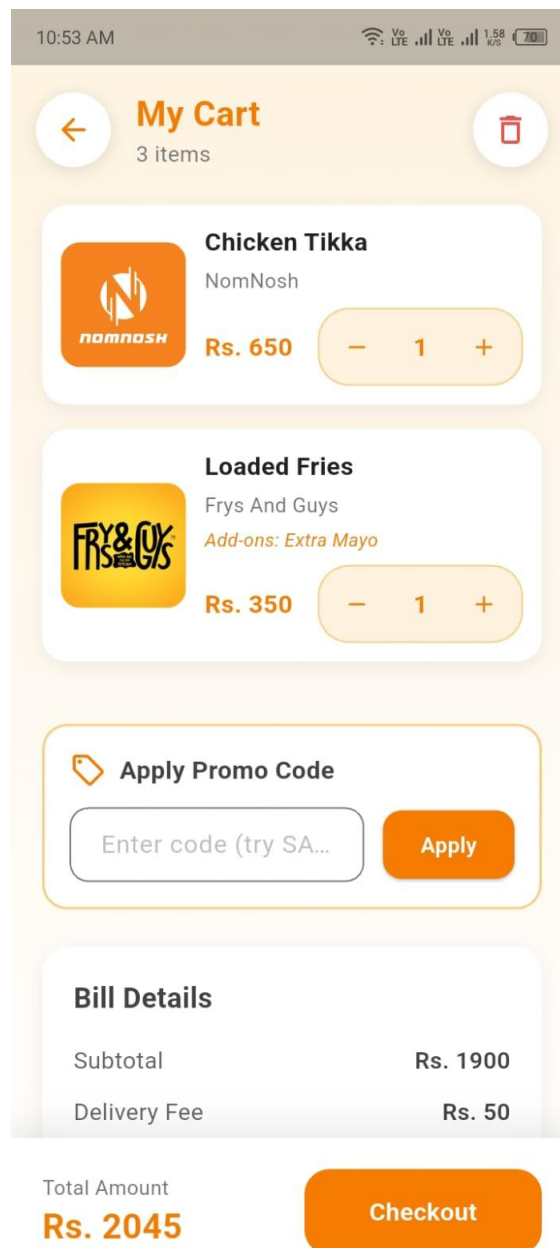


Figure 5.7 Cart Screen

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

Testing and evaluation were performed to check whether **Khaoo Piyoo** works according to the defined requirements and provides a smooth experience to users. Since the system includes multiple modules such as authentication, restaurant discovery, cart management, checkout, AI assistance, reels, profile management, and restaurant portal, testing was carried out at different levels to verify the behavior of each part.

The main purpose of testing was to identify errors, validate system functionality, and ensure that users can complete important tasks without difficulty. Both manual and automated testing approaches were considered. Manual testing was used to check screen navigation, form validation, button actions, order flow, and user interface behavior. Automated testing was used where repeated checks were required, such as input validation, login response, and basic functional verification.

The testing process focused on the following areas:

- User registration and login
- Role selection between customer and restaurant owner
- Restaurant search and browsing
- AI assistant response flow
- Reels viewing and interaction
- Cart item update and total calculation
- Checkout and order placement
- Profile and saved address management
- Restaurant portal login
- Password recovery

During evaluation, the system was tested from the perspective of both customers and restaurant owners. Customer-side testing verified whether users could search restaurants, add items to cart, apply promo codes, select delivery address, and place orders successfully. Restaurant-side testing checked login, registration, and access to business-related functions. Interface testing was also performed to ensure that screens remained consistent, readable, and easy to use.

The testing results showed that the core features of the system work properly under normal conditions. Minor issues related to layout spacing, form validation, and navigation flow were identified during testing and corrected during development. Overall, the evaluation confirms

that the system provides the required functionality and supports the main objectives of the project.

6.1 Manual Testing

Manual testing was performed to check the functionality of **Khaoo Piyoo** by interacting with the application as a normal user. Different features of the system were tested step by step to ensure that all operations work correctly and the system behaves as expected under normal conditions. The testing covered user-side features as well as restaurant-side access.

The process included checking registration, login, restaurant browsing, cart operations, checkout process, AI assistant interaction, reels viewing, and profile management. Each function was executed manually, and the system response was observed and compared with the expected outcome

The manual testing results show that the main features of the system are functioning correctly. Each test case was executed in real conditions, and the system produced the expected results. The navigation between screens is smooth, and the system responds properly to user inputs.

Table 6.1 Manual Testing Results

Test Case ID	Test Scenario	Expected Result	Status
TC-01	User Registration	Account created successfully	Pass
TC-02	User Login	User logs in with valid credentials	Pass
TC-03	Invalid Login	Error message displayed	Pass
TC-04	Role Selection	Correct role selected	Pass
TC-05	Restaurant Search	Relevant restaurants displayed	Pass
TC-06	Menu Viewing	Menu and prices displayed	Pass
TC-07	Add to Cart	Item added successfully	Pass
TC-08	Cart Update	Quantity updated correctly	Pass
TC-09	Checkout	Address and payment selected	Pass
TC-10	Place Order	Order confirmed successfully	Pass
TC-11	AI Assistant	Suggestions displayed	Pass

6.1.1 System testing

System testing was performed after the development of **Khaoo Piyoo** to ensure that the complete system works as intended and meets the defined requirements. In this phase, all

modules were tested together to verify that they interact correctly and provide the expected results. The purpose of system testing was to identify hidden errors, check overall performance, and ensure that the application is ready for deployment.

Different types of testing were carried out to evaluate the system from multiple perspectives. These include unit testing, functional testing, and integration testing. Each type of testing focuses on a specific aspect of the system and helps in improving the overall quality of the application.

Unit Testing: Individual components of the system, such as login, cart operations, and menu management, were tested separately to ensure that each function works correctly.

- **Functional Testing**

The system was tested according to its defined functionalities, such as restaurant search, order placement, AI recommendations, and profile management. This ensured that all features perform as expected.

- **Integration Testing**

Different modules were tested together to verify that data flows correctly between them. For example, adding items to the cart, proceeding to checkout, and placing an order were tested as a complete process.

System testing confirmed that the application behaves correctly under normal usage conditions. The main features of the system performed as expected, and the interaction between different modules remained consistent. Any issues identified during testing were resolved before final deployment to ensure a stable and reliable system for users.

6.1.2 Unit Testing

Unit testing was performed to check the working of individual components of **Khaoo Piyoo** before combining them with other modules. In this testing phase, each small function or screen was tested separately to confirm that it gives the correct output for the given input. This helped in identifying errors at an early stage and reduced issues during integration testing.

The major components tested during unit testing included login, registration, edit profile, cart update, password recovery, and restaurant search. Each test was performed with valid and invalid input values to verify that the system handles both successful and incorrect cases properly.

Table 6.2 Login Testing

No.	Test Case / Script	Input (Attribute Value)	Expected Result	Result
1	Click login button with correct credentials	Email/Username: valid userPassword: valid password	User should log in successfully and move to main page	Pass
2	Click login button with incorrect credentials	Email/Username: invalid userPassword: wrong password	Error message should be displayed	Pass
3	Click login button with empty fields	Empty email and password	Validation message should appear	Pass

Unit Testing 2: Edit Profile

Testing Objective: To ensure the edit profile form is working properly.

Table 6.3 Edit Profile Testing

No.	Test Case / Script	Input (Attribute Value)	Expected Result	Result
1	Update profile with valid data	Name, Email, Phone	Profile should update successfully	Pass
2	Submit profile form with empty fields	Missing required fields	Validation message should be shown	Pass
3	Update phone number with invalid format	Incorrect phone number	System should reject invalid input	Pass

6.1.3 Functional Testing

Functional testing was performed after unit testing to verify that each module of **Khaoo Piyoo** works according to the defined requirements. In this phase, the complete functionality of important modules was checked from the user's point of view. The main purpose was to confirm that the system performs the required operations correctly and displays the expected output.

Functional testing included checking different user roles, screen navigation, restaurant search, cart management, checkout, AI assistant, reels section, and profile management. Each test was performed using valid inputs to ensure that the system meets the required specifications.[10]

Functional Testing 1: Login with Different Roles

Objective: To ensure that the correct dashboard and navigation options are loaded according to the selected user role.

Table 6.4 Login with Different Roles

No.	Test Case / Script	Input (Attribute Value)	Expected Result	Result
1	Login as customer	Valid customer email and password	Customer home page is loaded with customer navigation options	Pass
2	Login as restaurant owner	Valid restaurant owner email and password	Restaurant dashboard is loaded with restaurant management options	Pass
3	Login as admin	Valid admin email and password	Admin dashboard is loaded with admin control options	Pass
4	Login with invalid role credentials	Invalid email or password	Error message is displayed and access is denied	Pass

Functional Testing 2: Restaurant Search and Menu Viewing

Objective: To ensure that users can search restaurants and view menu details correctly.

Table 6.5 Restaurant Search and Menu Viewing

No.	Test Case / Script	Input (Attribute Value)	Expected Result	Result
1	Search restaurant by name	Restaurant name	Matching restaurant is displayed	Pass
2	Search by food category	Category: Fast food / Desi / Pizza	Relevant restaurants are displayed	Pass
3	Open restaurant profile	Selected restaurant	Restaurant details and menu are displayed	Pass
4	View menu item price	Selected menu item	Item name, price, and details are shown	Pass

Functional Testing 3: Cart and Checkout

Objective: To ensure that the user can add items to cart and complete the checkout process.

Table 6.6 Cart and Checkout Testing

No.	Test Case / Script	Input (Attribute Value)	Expected Result	Result
1	Add item to cart	Selected food item	Item is added to cart	Pass
2	Update item quantity	Increase/decrease quantity	Cart total is updated correctly	Pass
3	Apply promo code	Valid promo code	Discount is applied	Pass
4	Proceed to checkout	Address and payment method	Order summary is displayed	Pass
5	Confirm order	Final order confirmation	Order is placed successfully	Pass

Functional Testing 4: AI Assistant and Reels

Objective: To ensure that engagement and recommendation features work properly.

Table 6. 7 AI Assistant and Reels Testing

No.	Test Case / Script	Input (Attribute Value)	Expected Result	Result
1	Ask AI assistant for food suggestion	“Suggest nearby pizza”	Relevant food or restaurant suggestion is displayed	Pass
2	Ask budget-based query	“Food under 500”	Budget-friendly options are displayed	Pass
3	Open reels section	Tap reels button	Food reels are loaded	Pass
4	Interact with reel	Like/comment/share	Selected action is performed	Pass

6.1.4 Automated Testing

Automated testing was performed to verify important functionalities of **Khaoo Piyoo** where repeated execution and consistent results were required. Automated tools were used to check input validation, authentication flow, API responses, and basic UI behavior. This helped in

reducing manual effort and ensured that key system functions remain stable during development.

The testing tools were applied on selected functional requirements such as login validation, restaurant search, API response handling, and non-functional requirements like performance and reliability. Automated testing also helped in quickly identifying issues during updates and ensuring that previously working features were not affected.

This table show Automated testing results indicate that the system performs consistently for repeated operations and critical functionalities. API responses, authentication processes, and validation checks worked as expected. The use of automated tools helped ensure accuracy and stability of the system while reducing the chances of human error during testing.[13]

Table 6.8 Automated Testing Tools and Results

Tool Name	Applied on (Test Cases / FR / NFR)	Results
Flutter Test	Login, Registration, Form Validation	All validation tests passed successfully
Postman	API Testing (Restaurant data, Orders, User data)	APIs returned correct responses
Firebase Test Lab	App performance and device compatibility	Application performed well on tested devices
JUnit / Unit Test Framework	Core logic (Authentication, Cart calculation)	All test cases executed successfully
Integration Test (Flutter)	Navigation flow, Cart to Checkout process	Flow executed without errors

Chapter 7

Conclusion

7 Conclusion and Future Work

The development of **Khaoo Piyoo** successfully addresses the need for a centralized food discovery and ordering platform. The system brings important features such as restaurant browsing, menu viewing, price comparison, location-based search, AI-based assistance, reels engagement, cart management, and order placement into a single application. This helps users explore available food options more easily and make informed decisions according to their preferences and budget.

The system also provides value to restaurant owners by giving them a digital platform where they can register their business, manage menus, upload offers, promote food items, and reach more customers. The role-based structure of the application ensures that customers, restaurant owners, and administrators can access features according to their needs.

Testing results show that the core modules of the system work according to the defined requirements. Features such as login, restaurant search, cart update, checkout, AI assistant, reels section, and profile management were tested and produced the expected results. Overall, the project provides a practical, user-friendly, and scalable solution for improving restaurant discovery and food ordering.

Although the current system fulfills its main objectives, several improvements can be added in future versions to make the platform more advanced and complete. The future enhancements may include online payment integration, real-time delivery tracking, improved AI-based recommendations, and a more detailed analytics dashboard for restaurant owners.

The system can also be expanded by adding multilingual support, advanced filtering options, customer review analysis, and better promotion tools for restaurants. In future updates, the AI assistant can be improved to learn from user history and provide more accurate meal and restaurant suggestions.[14]

Possible future enhancements include:

- Online payment gateway integration
- Real-time order and delivery tracking
- Advanced AI recommendation system
- Restaurant analytics dashboard
- Multilingual support
- Customer review and rating analysis
- Improved security and privacy features

- Web-based version of the application
- Delivery rider module
- Advanced promotional campaign system

These future improvements can increase the overall usefulness, performance, and scalability of **Khaoo Piyoo**, making it more suitable for a larger audience and long-term use.

7.1 Conclusion

The development of **Khaoo Piyoo** provides a complete and practical solution for improving the process of restaurant discovery and food selection. The system brings together essential features such as restaurant browsing, menu viewing, price comparison, location-based search, AI-based assistance, reels engagement, and order placement within a single platform. This integration helps users make informed decisions quickly and reduces the need to rely on multiple applications.

The system also supports restaurant owners by providing a platform to register their business, manage menus, promote offers, and reach a wider audience. The role-based structure ensures that each type of user can access relevant features, making the system organized and efficient. Throughout the development process, the system was designed and implemented according to the defined requirements. Testing results confirmed that the major functionalities operate correctly and provide a stable user experience. The modular design and system architecture also allow future enhancements without affecting the overall performance.

Overall, the project achieves its objective of creating a user-friendly, reliable, and scalable application that improves accessibility and transparency in the food industry.

7.2 Future Work

Although **Khaoo Piyoo** provides the required functionality for restaurant discovery and ordering, there are several areas where the system can be further improved and expanded. These enhancements can increase the efficiency of the system, improve user experience, and make the platform more competitive in the market.

One of the important improvements is the integration of a secure online payment system. Currently, the system focuses on order placement, but adding multiple payment options such as debit/credit cards, mobile wallets, and digital payment gateways will make transactions more convenient and reliable for users. In addition, real-time order tracking can be introduced so that users can monitor the status of their orders from preparation to delivery.

The AI assistant can also be enhanced by using advanced recommendation techniques. By analyzing user history, preferences, and behavior, the system can provide more accurate and personalized suggestions. This will improve user satisfaction and make the application more intelligent over time.

Another important area for improvement is the development of a dedicated delivery module. This would include features for delivery riders, such as order assignment, route navigation, and delivery status updates. This addition would make the system more complete by covering the entire food ordering process from selection to delivery.

The system can also be improved by introducing a detailed analytics dashboard for restaurant owners. This dashboard can provide insights such as order trends, popular menu items, customer behavior, and revenue reports. These insights will help restaurant owners make better business decisions.

To increase accessibility, multilingual support can be added so that users from different regions can use the application in their preferred language. Similarly, a web-based version of the system can be developed to allow access through desktop browsers in addition to mobile devices.

Further improvements may include better security mechanisms, advanced filtering options for restaurant search, enhanced user review systems, and improved promotional tools for restaurants. Features such as personalized offers, seasonal discounts, and targeted advertisements can also be introduced to increase user engagement.[15]

Some key future enhancements include:

- Integration of secure online payment methods
- Real-time order and delivery tracking
- Advanced AI-based recommendation system
- Delivery rider and logistics management module
- Restaurant analytics and reporting dashboard
- Multilingual support for wider accessibility
- Web-based version of the system
- Enhanced security and data protection
- Improved filtering and search options
- Advanced promotional and marketing features

These improvements can make **Khaoo Piyou** more efficient, scalable, and user-friendly, while also supporting long-term growth and adaptability of the system.

References

- [1] Flutter, “Flutter Documentation,” 2024. [Online]. Available: <https://docs.flutter.dev>
- [2] Firebase, “Firebase Documentation,” 2024. [Online]. Available: <https://firebase.google.com/docs>
- [3] Google Developers, “Google Maps Platform Documentation,” 2024. [Online]. Available: <https://developers.google.com/maps/documentation>
- [4] Google Cloud, “Dialogflow Documentation,” 2024. [Online]. Available: <https://cloud.google.com/dialogflow/docs>
- [5] Dart Team, “Dart Programming Language Documentation,” 2024. [Online]. Available: <https://dart.dev/guides>
- [6] Android Developers, “Android Development Documentation,” 2024. [Online]. Available: <https://developer.android.com>
- [7] Apple Inc., “iOS Developer Documentation,” 2024. [Online]. Available: <https://developer.apple.com/documentation>
- [8] Postman Inc., “Postman API Platform Documentation,” 2024. [Online]. Available: <https://learning.postman.com/docs>
- [9] GitHub, “GitHub Documentation,” 2024. [Online]. Available: <https://docs.github.com>
- [10] OpenAI, “OpenAI API Documentation,” 2024. [Online]. Available: <https://platform.openai.com/docs>
- [11] Microsoft, “Visual Studio Code Documentation,” 2024. [Online]. Available: <https://code.visualstudio.com/docs>
- [12] Google, “Firebase Cloud Messaging Documentation,” 2024. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging>
- [13] Google, “Cloud Firestore Documentation,” 2024. [Online]. Available: <https://firebase.google.com/docs/firestore>
- [14] Google, “Firebase Authentication Documentation,” 2024. [Online]. Available: <https://firebase.google.com/docs/auth>
- [15] Google, “Firebase Storage Documentation,” 2024. [Online]. Available: <https://firebase.google.com/docs/storage>